

Corso di Architettura degli Elaboratori a.a. 2025/2026

L'Instruction Set Architecture (ISA) RISC-V

Le procedure

- Porzioni di codice associate ad un **nome** che possono essere invocate più volte e che eseguono un compito specifico, avendo come input una lista di **parametri** e come output un **valore di ritorno**
- Vantaggi
 - Astrazione
 - Riutilizzabilità del codice (librerie)
 - Maggiore organizzazione del codice
 - Testing più agevole

```
int somma(int x, int y)
{
    int rst;
    rst = x + y;
    return rst;
}
```

Procedure: un esempio

Programma (procedura)
chiamante

```
...  
f=f+1;  
risultato = somma(f,g);  
...
```

Procedura
chiamata

```
int somma(int x, int y)  
{  
    int rst;  
    rst = x + y + 2;  
    return rst;  
}
```

Le procedure: passi da seguire

- Chiamante
 - Mettere i parametri in un luogo accessibile alla procedura
 - Trasferire il controllo alla procedura
 - Chiamato
 - Acquisire le risorse necessarie per l'esecuzione della procedura
 - Eseguire il compito richiesto
 - Mettere il risultato in un luogo accessibile al programma chiamante
 - Restituire il controllo al punto di origine
- (la stessa procedura può essere chiamata in differenti punti di un programma)**

Modifica del flusso di programma: invocazione

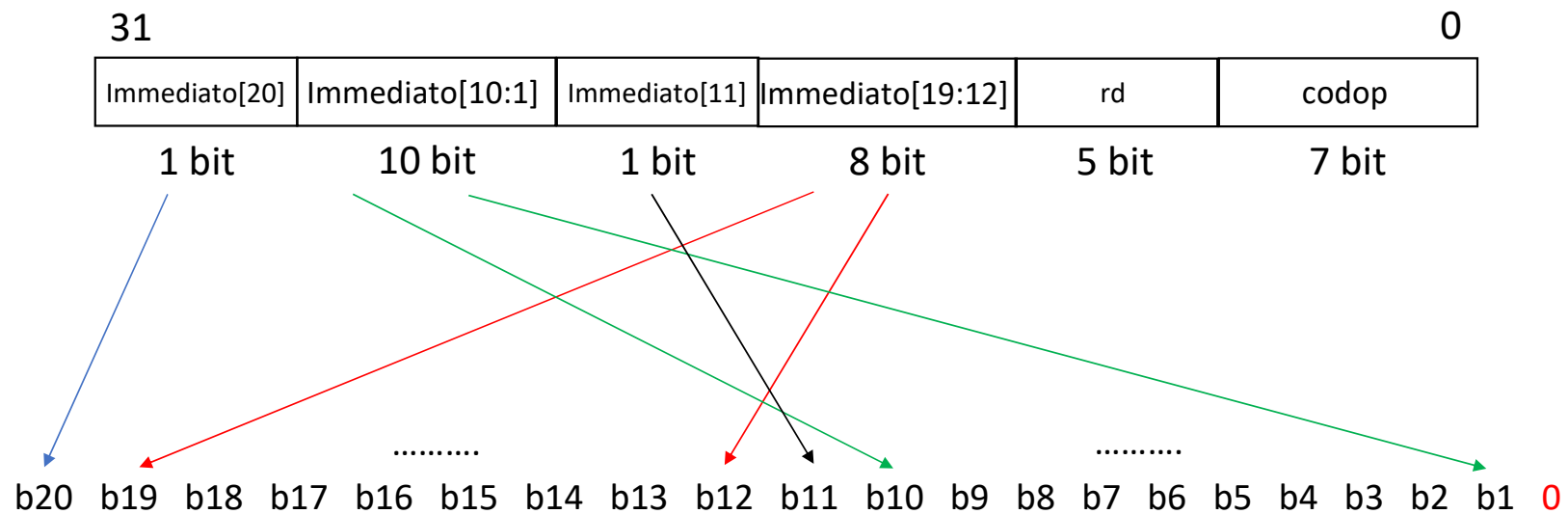
```
jal rd, IndirizzoProcedura      Jump-and-link
```

- Salta all'indirizzo (offset) con etichetta `IndirizzoProcedura`
- Memorizza l'indirizzo dell'istruzione successiva `PC+4` nel registro `rd`
- Per `rd` noi useremo sempre e solo `x1` ovvero il suo alias **ra** (return address)
- Pseudo-istruzione

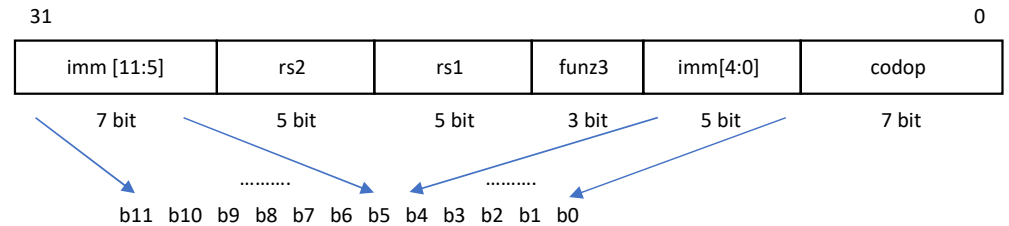
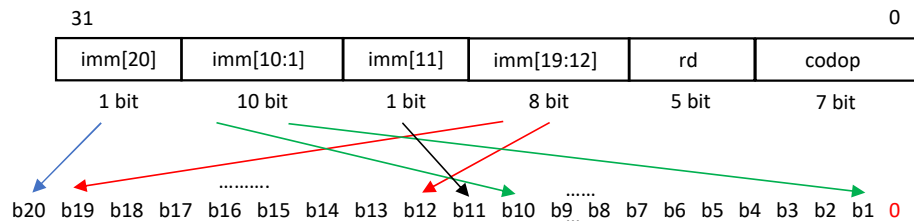
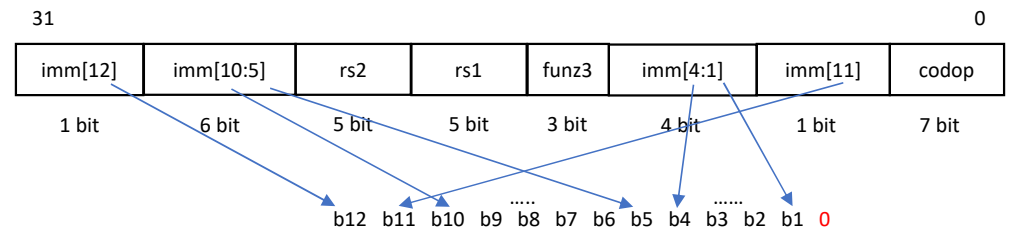
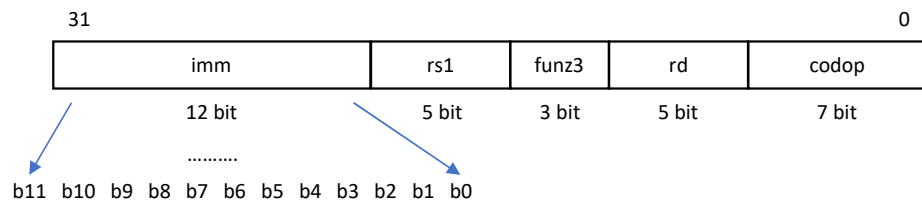
```
jal IndirizzoProcedura
```

JAL e linguaggio macchina

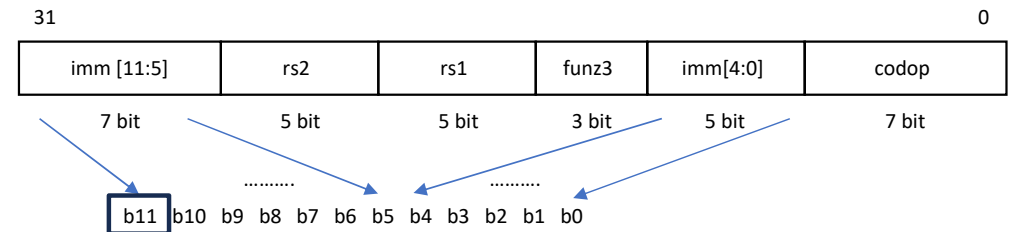
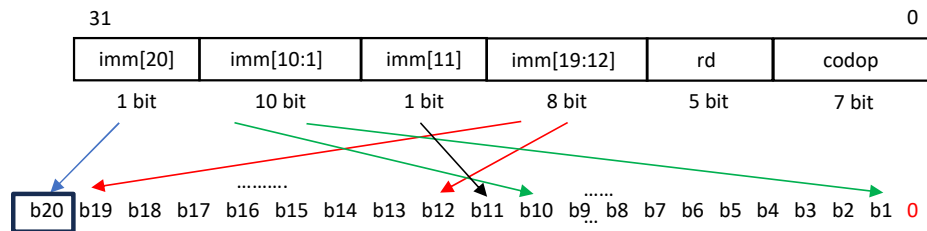
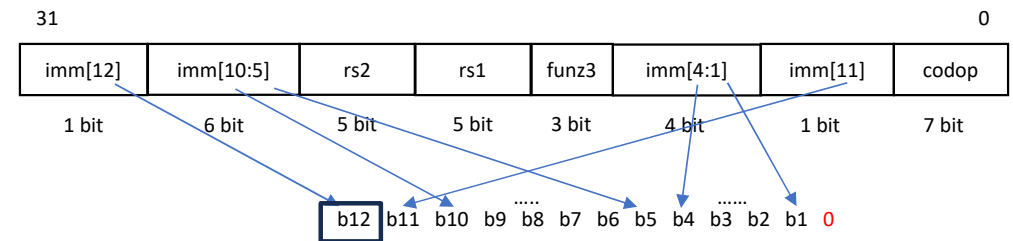
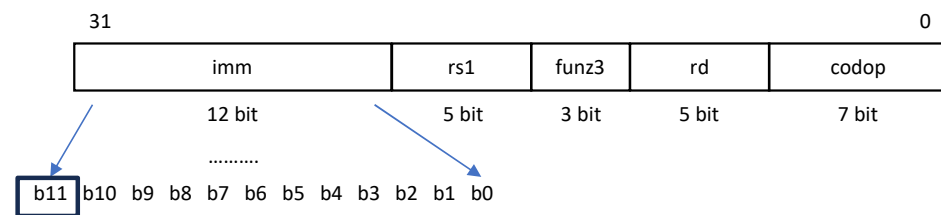
- Viene introdotto un nuovo tipo: J



Cosa hanno in comune i formati I, S, SB e J?



Cosa hanno in comune i formati I, S, SB e J?



Il bit di segno dell'immediato contenuto nell'istruzione è SEMPRE il b31 dell'istruzione (importante per il circuito di estensione del segno).

Modifica del flusso di programma: ritorno al chiamante

Jump-and-link register

```
jalr rd, offset(rs1)
```

- Salta ad un indirizzo qualsiasi

- $x[rd] = PC + 4; PC = x[rs1] + \text{sext}(\text{offset}) \& \sim 1$

Segno esteso a 32 bit in RV32I (64 in RV64I)

Bit 0 azzerato

- La procedura chiamata come ultima istruzione esegue

Pseudoistruzione

```
jr ra
```

```
jalr x0, 0(x1)
```

Tipo I in linguaggio macchina

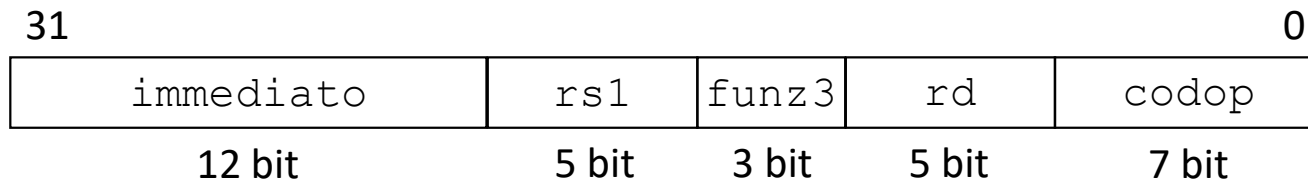
$x0 = PC + 4; PC = x1 + 0$

Operazione nulla

Return Address

RISC-V Instruction Set

Formato di tipo I (immediato)



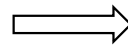
- Permette di codificare anche jalr
- Il campo immediato
 - Rappresentato in complemento a due
 - Valori possibili: da -2048 a $+2047$

Side effect – sovrascrittura dei registri (1)

```
int somma(int x, int y) {  
    int rst;  
    rst = x + y + 2;  
    return rst;  
}
```

```
...  
f=f+1  
risultato=somma(f,g)  
...
```

x → x10
y → x11
rst → x20
f → x6



```
SOMMA: add  x5,x10,x11  
        addi x20,x5,2  
        jalr x0,0(x1)  
...  
...  
        addi x6,x6,1  
...  
        jal  SOMMA
```

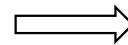
- **Problema:** che cosa succede se il registro x5 contiene un valore usato dalla procedura chiamante?
- **Soluzione:**

Side effect – sovrascrittura dei registri (1)

```
int somma(int x, int y) {  
    int rst;  
    rst = x + y + 2;  
    return rst;  
}
```

```
...  
f=f+1  
risultato=somma(f,g)  
...
```

x → x10
y → x11
rst → x20
f → x6

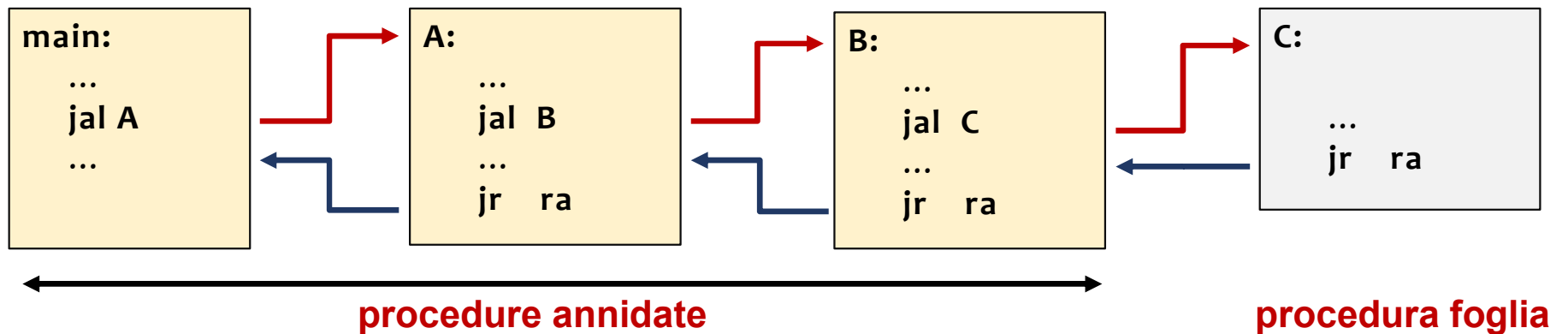


```
SOMMA: add  x5,x10,x11  
        addi x20,x5,2  
        jalr x0,0(x1)  
...  
...  
        addi x6,x6,1  
...  
        jal  SOMMA
```

- **Problema:** che cosa succede se il registro x5 contiene un valore usato dalla procedura chiamante?
- **Soluzione:** **salvare** il valore di x5 **in memoria** prima di utilizzarlo (e ripristinarlo prima del ritorno al chiamante)

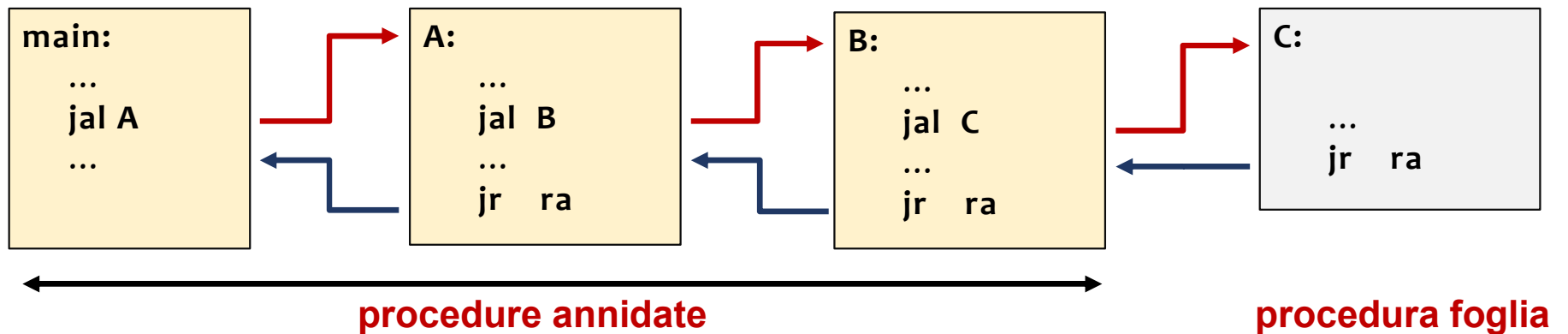
Side effect – procedure annidate (2)

- Problema
 - Nel caso di procedure annidate, il return address (`x1` o `ra`) viene sovrascritto
- Soluzione:



Side effect – procedure annidate (2)

- Problema
 - Nel caso di procedure annidate, il return address (`x1` o `ra`) viene sovrascritto
- Soluzione:
 - La procedura chiamata, deve **salvare in memoria** il valore di `x1` **prima** di chiamare la procedura annidata con l'istruzione `jal`



Side effect – parametri numerosi (3)

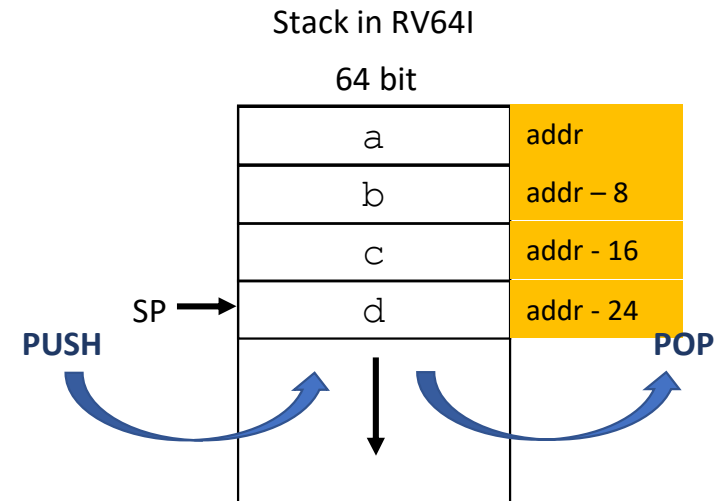
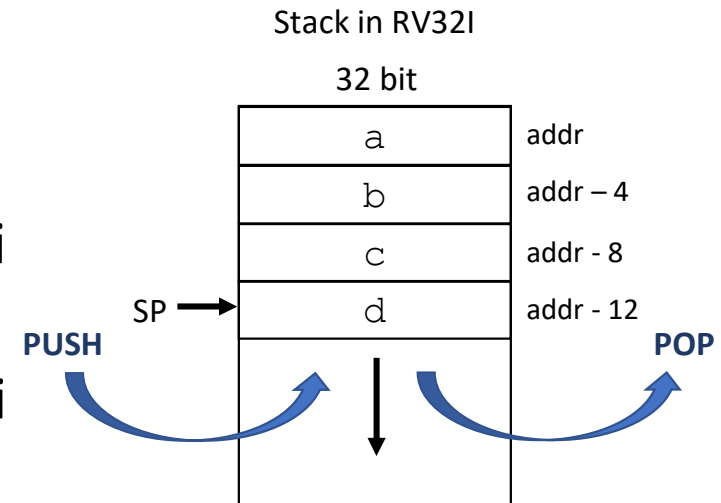
- Problema:
 - che cosa succede se i parametri e le variabili di una procedura superano il numero di registri disponibili?
- Soluzione:

Side effect – parametri numerosi (3)

- Problema:
 - che cosa succede se i parametri e le variabili di una procedura superano il numero di registri disponibili?
- Soluzione:
 - **Salvare** temporaneamente i dati **in memoria** per caricarli nei registri **prima** del loro utilizzo all'interno della procedura

Lo stack

- In quale area di memoria è possibile salvare i registri per evitarne la perdita del valore?
- In memoria viene definita una struttura dati dinamica, lo stack
- Accesso: Last In First Out (LIFO)
- Operazioni:
 - PUSH: aggiunge un elemento «in cima allo stack»
 - POP: rimuove un elemento «dalla cima dello stack»
 - La cima dello stack è identificata dallo Stack Pointer (SP)



Lo stack

- In RISC V si usa la convenzione
 - **grow-down**: lo stack cresce da indirizzi di memoria alti verso indirizzi di memoria bassi
 - **last-full**: lo stack pointer (SP) contiene l'indirizzo dell'ultima cella di memoria occupata nello stack
 - Il valore di SP è salvato nel registro x2 (a.k.a. `sp`)

- **PUSH**

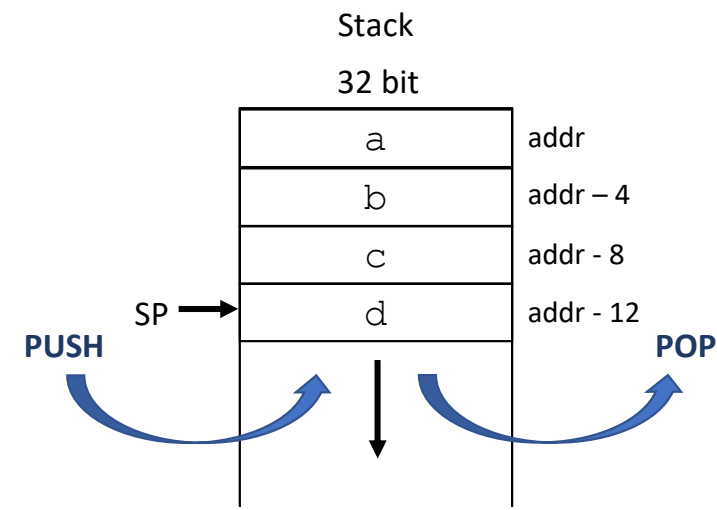
- Decrementa SP
- Scrive in `M[SP]`

```
addi sp, sp, -4
sw    x20, 0(sp)
```

- **POP**

- Legge da `M[SP]`
- Incrementa SP

```
lw    x20, 0(sp)
addi sp, sp, 4
```



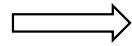
ESERCIZIO: modificare PUSH e POP per RV64I

Nell'esempio di prima:

```
int somma(int x, int y) {  
    int rst;  
    rst = x + y + 2;  
    return rst;  
}
```

```
...  
f=f+1  
risultato=somma(f,g)  
...
```

x → x10
y → x11
rst → x20
f → x6



```
SOMMA: addi sp,sp,-8  
       sw   x5,0(sp)  
       sw   x20,4(sp)  
       add  x5,x10,x11  
       addi x20,x5,2  
       addi x10,x20,0  
       lw   x5,0(sp)  
       lw   x20,4(sp)  
       addi sp,sp,8  
       jalr x0,0(x1)
```

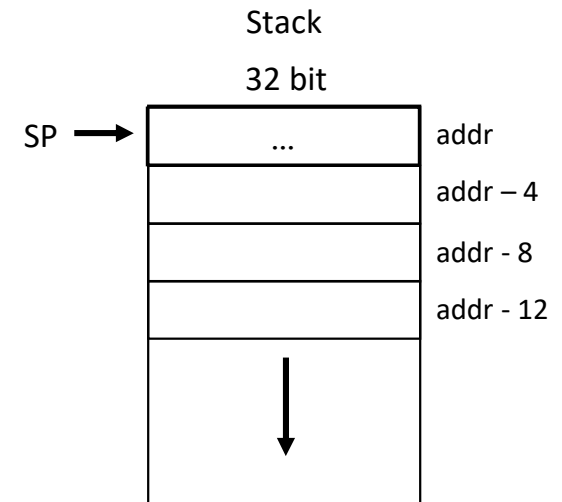
...

...

```
       addi x6,x6,1
```

...

```
       jal  SOMMA
```

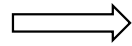


Nell'esempio di prima:

```
int somma(int x, int y) {  
    int rst;  
    rst = x + y + 2;  
    return rst;  
}
```

```
...  
f=f+1  
risultato=somma(f,g)  
...
```

x → x10
y → x11
rst → x20
f → x6



```
SOMMA: addi sp,sp,-8  
       sw   x5,0(sp)  
       sw   x20,4(sp)  
       add  x5,x10,x11  
       addi x20,x5,2  
       addi x10,x20,0  
       lw   x5,0(sp)  
       lw   x20,4(sp)  
       addi sp,sp,8  
       jalr x0,0(x1)
```

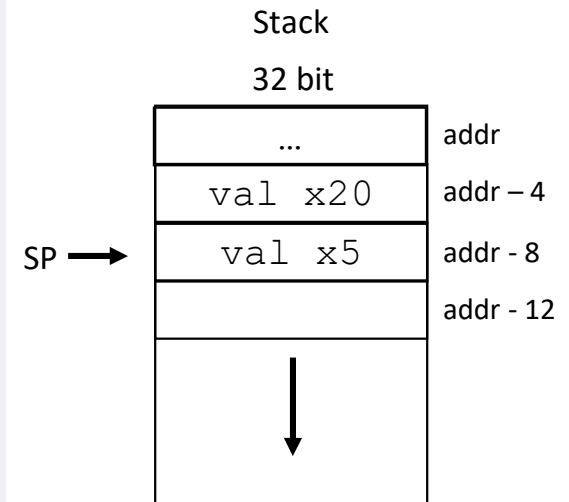
...

...

```
       addi x6,x6,1
```

...

```
       jal  SOMMA
```



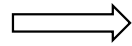
**PUSH: salvataggio del valore
di x5 e x20**

Nell'esempio di prima:

```
int somma(int x, int y) {  
    int rst;  
    rst = x + y + 2;  
    return rst;  
}
```

```
...  
f=f+1  
risultato=somma(f,g)  
...
```

x → x10
y → x11
rst → x20
f → x6



```
SOMMA: addi sp, sp, -8  
       sw   x5, 0(sp)  
       sw   x20, 4(sp)  
       add   x5, x10, x11  
       addi  x20, x5, 2  
       addi  x10, x20, 0  
       lw   x5, 0(sp)  
       lw   x20, 4(sp)  
       addi sp, sp, 8  
       jalr x0, 0(x1)
```

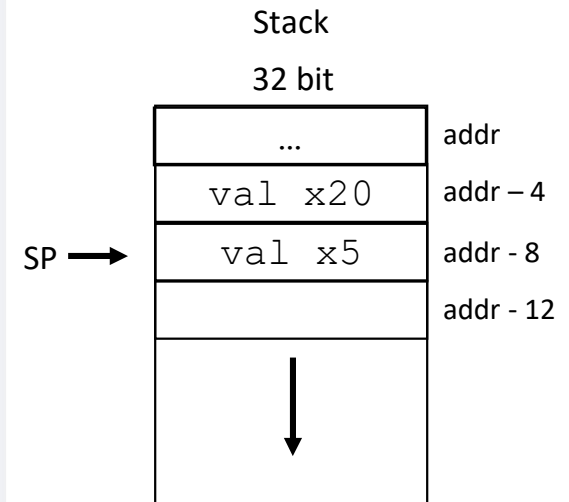
...

...

```
addi x6, x6, 1
```

...

```
jal SOMMA
```



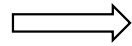
Corpo della procedura

Nell'esempio di prima:

```
int somma(int x, int y) {  
    int rst;  
    rst = x + y + 2;  
    return rst;  
}
```

```
...  
f=f+1  
risultato=somma(f,g)  
...
```

x → x10
y → x11
rst → x20
f → x6



```
SOMMA: addi sp, sp, -8  
       sw   x5, 0(sp)  
       sw   x20, 4(sp)  
       add  x5, x10, x11  
       addi x20, x5, 2  
       addi x10, x20, 0  
       lw   x5, 0(sp)  
       lw   x20, 4(sp)  
       addi sp, sp, 8  
       jalr x0, 0(x1)
```

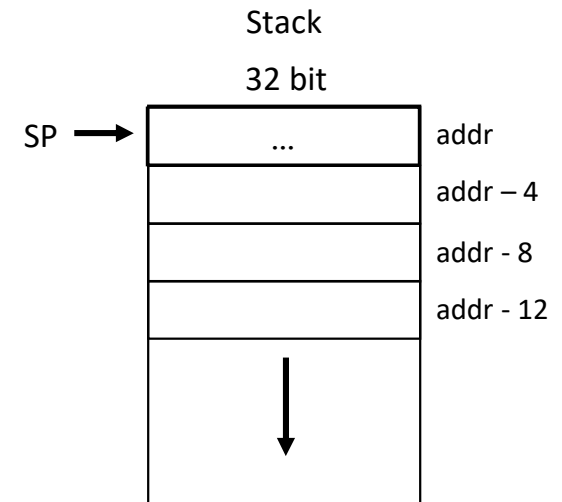
...

...

```
addi x6, x6, 1
```

...

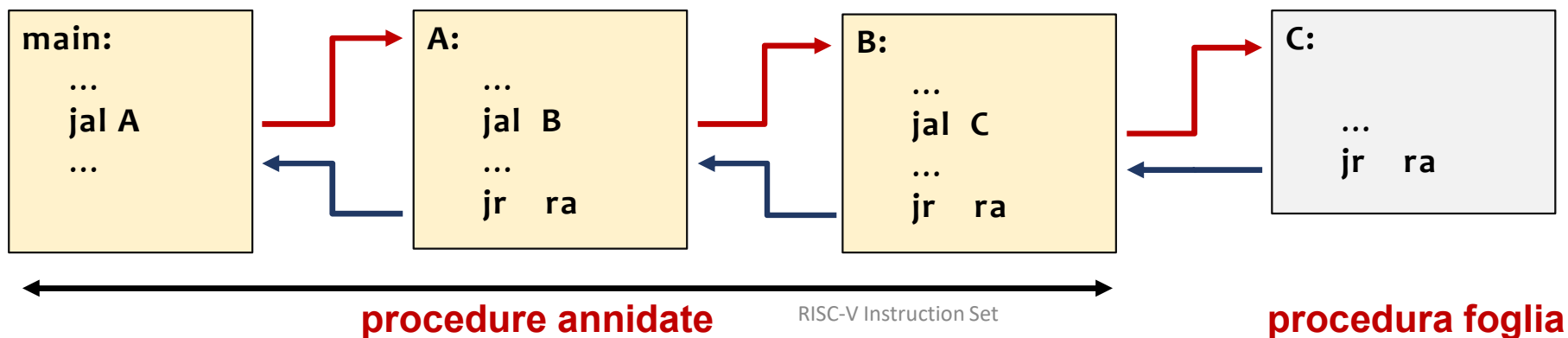
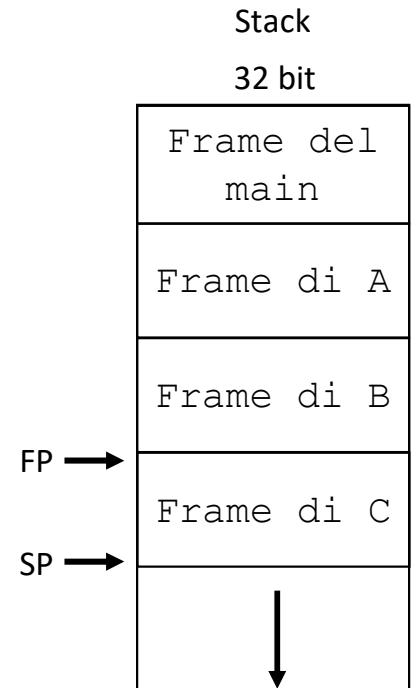
```
jal SOMMA
```



**POP: ripristino del valore
di x5 e x20**

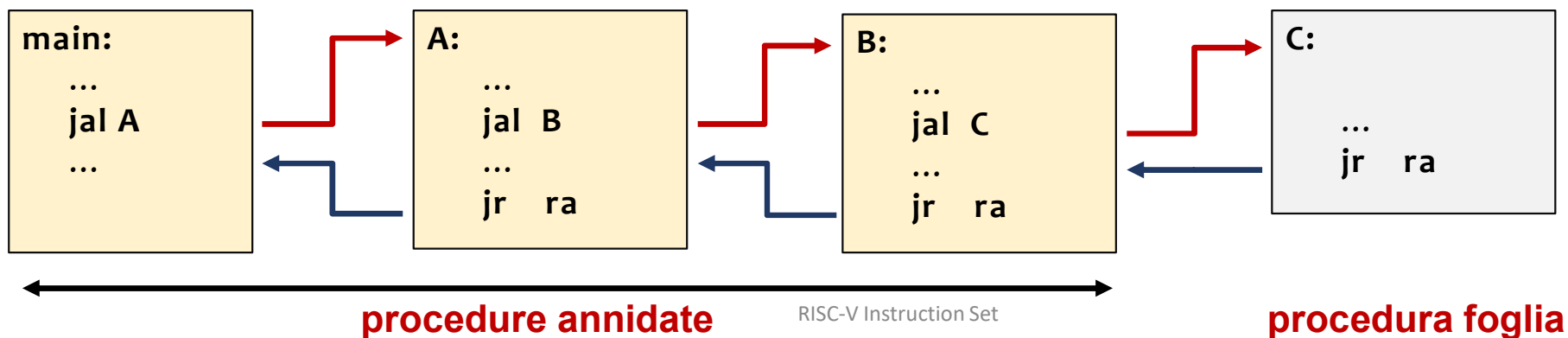
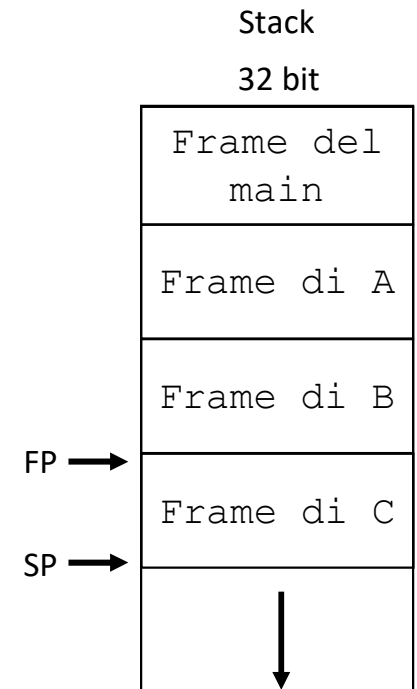
Record di attivazione

- Ad ogni esecuzione di procedura corrisponde un blocco di memoria all'interno dello stack che viene chiamato **record di attivazione o frame**
- Quando viene richiamata la funzione F, viene aggiunto un record di attivazione allo stack
- Quando la funzione F termina, il corrispondente record di attivazione viene cancellato



Record di attivazione

- Il frame in cima allo stack rappresenta il frame della procedura in esecuzione
- I confini del frame sono identificati da
 - Base: il Frame Pointer (registro x8, fp)
 - Cima: lo Stack Pointer (registro x2, sp)



I registri e convenzioni sul loro uso

Registro	Nome	Utilizzo
x0	zero	Costante zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Puntatore a thread
x8	s0 / fp	Frame pointer (il contenuto va preservato se utilizzato dalla procedura chiamata)
x10-x11	a0-a1	Passaggio di parametri nelle procedure e valori di ritorno
x12-x17	a2-a7	Passaggio di parametri nelle procedure
x5-x7 x28-x31	t0-t2 t3-t6	Registri temporanei, non salvati in caso di chiamata
x9	s1	Registri da salvare: il contenuto va preservato se utilizzati dalla procedura chiamata
x18-x27	s2-s11	

Convenzione nell'uso e salvataggio dei registri

- Chi è responsabile di salvare i registri quando si effettuano chiamate di funzioni?
 - La funzione chiamante conosce quali registri sono importanti per sé e che dovrebbero essere salvati
 - La funzione chiamata conosce quali registri userà e che dovrebbero essere salvati prima di modificarli
- Bisogna evitare le inefficienze → Minimo salvataggio dei registri:
 - La funzione chiamante potrebbe salvare tutti i registri che sono importanti per sé, anche se la procedura chiamata non li modificherà
 - La funzione chiamata potrebbe salvare tutti i registri che si appresta a modificare, anche quelli che non verranno poi utilizzati dalla procedura chiamante una volta che la procedura chiamata le avrà restituito il controllo

Convenzione nell'uso e salvataggio dei registri

- Necessità di stabilire delle convenzioni
- I registri $x10-x17$ ($a0-a7$), $x5-x7$ e $x28-x31$ ($t0-t6$)
 - possono essere modificati dal chiamato senza nessun meccanismo di ripristino
 - Il chiamante se necessario dovrà salvare i valori dei registri prima dell'invocazione della procedura
- I registri $x1$ (ra), $x2$ (sp), $x3$ (gp), $x4$ (tp), $x8$ ($fp/s0$), $x9$ e $x18-x27$ ($s1-s11$)
 - Se modificati dal chiamato devono essere salvati e poi ripristinati prima del ritorno al chiamante
 - Il chiamante non è tenuto al loro salvataggio e ripristino

Convenzioni nell'uso e salvataggio dei registri: perchè?

- Si supponga che
 - La funzione chiamante abbia bisogno del valore dei registri $s1, s2, s3$ dopo l'invocazione di una funzione f
 - La funzione chiamata f abbia bisogno di modificare i valori dei registri $t0$ e $t1$
- Quale convenzione è la più vantaggiosa?

Convenzioni nell'uso e salvataggio dei registri: perchè?

1 - Nessuna convenzione: chiamante e chiamato si devono preoccupare di salvare e ripristinare i registri che gli interessano

- Chiamante: 3 salvataggi e ripristini ($s1$, $s2$, $s3$)
- Chiamato: 2 salvataggi e ripristini ($t0$, $t1$)

2 - Il chiamante si deve preoccupare di salvare i registri che intende preservare dopo l'invocazione di f

- Chiamante: 3 salvataggi e ripristini ($s1$, $s2$, $s3$)
- Chiamato: Nessun salvataggio

Convenzioni nell'uso e salvataggio dei registri: perchè?

3 - Il chiamato si deve preoccupare di salvare i registri che intende modificare durante l'esecuzione di f

- Chiamante: nessun salvataggio e ripristino
- Chiamato: 2 salvataggi e ripristini (t_0 , t_1)

4 - Convenzione RISC-V

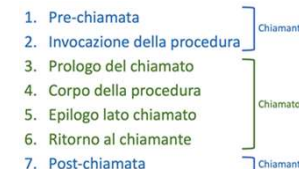
- Chiamante: nessun salvataggio e ripristino
- Chiamato: nessun salvataggio e ripristino

Le fasi di una invocazione di procedura

Possiamo dividere l'invocazione di una procedura in 7 fasi:

-
- 1. Pre-chiamata
 - 2. Invocazione della procedura
 - 3. Prologo del chiamato
 - 4. Corpo della procedura
 - 5. Epilogo lato chiamato
 - 6. Ritorno al chiamante
 - 7. Post-chiamata
- The diagram uses brackets to group the phases into three categories:
- Chiamante (Caller):** Indicated by a blue bracket on the right, it groups phases 1 and 2.
 - Chiamato (Callee):** Indicated by a green bracket on the right, it groups phases 3, 4, and 5.
 - Chiamante (Caller):** Indicated by a blue bracket on the right, it groups phases 6 and 7.

Le fasi di una invocazione di procedura



Fase 1 – Pre-chiamata del chiamante

- Eventuale salvataggio registri da preservare nel chiamante
 - Si assume che $x_{10}-x_{17}$ (a_0-a_7), x_5-x_7 e $x_{28}-x_{31}$ (t_0-t_6), possano essere sovrascritti dal chiamato
 - se li si vuole preservare vanno salvati nello stack (dal chiamante) – vedi caso 1
 - il caso 2 mostra un caso in cui non è necessario salvare il contenuto del registro associato alla variabile f
- Preparazione degli argomenti della funzione
 - I primi 8 argomenti vengono posti in $x_{10}-x_{17}$, ovvero a_0-a_7 (nuovi valori)
 - Gli eventuali altri argomenti oltre l'ottavo vanno salvati nello stack (`EXTRA_ARGS`), così che si trovino subito sopra il frame della funzione chiamata

```
int somma(int x, int y) {  
    x=x+y;  
    return x;  
}  
...  
f=f+1;  
risultato=somma(f,g);  
printf("%d", f);  
return;
```

1

```
int somma(int x, int y) {  
    x=x+y;  
    return x;  
}  
...  
f=f+1;  
risultato=somma(f,g);  
return;
```

2

Le fasi di una invocazione di procedura



Fase 2 – Invocazione della procedura

- Istruzione `jal NOME_PROCEDURA`

Fase 3 – Prologo lato chiamato

- Eventuale allocazione del call-frame sullo stack (aggiornare `sp`)
 - Salvataggio di `x1 (ra)` nel caso in cui la procedura non sia foglia
 - Salvataggio di `x8 (fp)`, solo se utilizzato all'interno della procedura
 - Salvataggio di `x9` e `x18–x27 (s1–s11)` se utilizzati all'interno della procedura (il chiamante si aspetta di trovarli intatti)
 - Salvataggio degli argomenti `x10–x17 (a0–a7)` solo se la funzione li riuserà successivamente a ulteriori chiamate a funzione (procedure annidate) (nota: `x10–x17` sono trattati come i registri temporanei – possono essere sovrascritti)
- Eventuale inizializzazione di `fp`: punta al nuovo call-frame

Le fasi di una invocazione di procedura



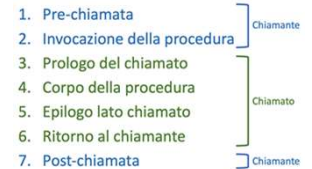
Fase 4 – Corpo della procedura

- Istruzioni che implementano il corpo della procedura

Fase 5 – Epilogo lato chiamato

- Se deve essere restituito un valore dalla funzione
 - Tale valore viene posto in `x10` (e `x11`) ovvero `a0-a1`
- I registri (se salvati) devono essere ripristinati
 - `x9` e `x18-x27` (`s1-s11`)
 - `x1` (`ra`)
 - `x8` (`fp`)
- Notare che `sp` deve solo essere aumentato di opportuno offset (lo stesso sottratto nella Fase 3)

Le fasi di una invocazione di procedura



Fase 6 – Ritorno al chiamante

- Istruzione `jalr x0, 0(x1)` (o pseudo-istruzione `jr ra`)

Fase 7 – Post-chiamata lato chiamante

- Eventuale uso del risultato della funzione (in `x10` e `x11`, cioè `a0-a1`)
- Ripristino dei valori `x5-x7` e `x28-x31` (`t0-t6`), `x10-x17` (`a0-a7`) vecchi, eventualmente salvati

Le fasi di una invocazione di procedura foglia



Le fasi descritte precedentemente si applicano a tutti i tipi di procedura, ma possono essere semplificate:

- Se una procedura è foglia, nel codice della procedura
 - non è necessario salvare e ripristinare il registro ra
 - non è necessario salvare sullo stack i registri $a0-a7$ e $t0-t6$ (non c'è nessuna invocazione di procedura nella procedura chiamata)

Le fasi di una invocazione di procedura foglia



Fase 1 – Pre-chiamata del chiamante

- Eventuale salvataggio registri da preservare nel chiamante
 - x10-x17 (a0-a7), x5-x7 e x28-x31 (t0-t6), potrebbero essere sovrascritti dal chiamato
 - se li si vuole preservarli vanno salvati nello stack (dal chiamante)
- Preparazione degli argomenti della funzione
 - I primi 8 argomenti vengono posti in x10-x17, ovvero a0-a7 (nuovi valori)
 - Gli eventuali altri argomenti oltre l'ottavo vanno salvati nello stack (EXTRA_ARGS), così che si trovino subito sopra il frame della funzione chiamata

```
sommaArray:
    li t0, 0          # t0 = somma
    li t1, 0          # t1 = contatore indice i
loop:
    bge t1, a1, fine  # i >= lunghezza, esci dal loop
    slli t2, t1, 2    # offset
    add t3, a0, t2    # indirizzo elemento i
    lw t4, 0(t3)      # Carica v[i]
    add t0, t0, t4    # Aggiorna la somma
    addi t1, t1, 1    # i++
    j loop            # Ripeti il ciclo

fine:
    mv a0, t0         # Restituisci il risultato in a0
    jr ra             # Ritorna

# Funzione main
start:
    la a0, array      # Indirizzo dell'array in a0
    la a1, length     # Lunghezza dell'array in a1
    lw a1, 0(a1)
    jal ra, sommaArray

    li a7, 93         # syscall exit (codice in a0)
    ecall             # Termina il programma
```

Le fasi di una invocazione di procedura foglia



Non serve salvare `ra`

In questo esempio non serve salvare registri di `s1-s11`

In questo esempio non serve modificare il registro `sp`

Fase 2 – Invocazione della procedura

- Istruzione `jal NOME_PROCEDURA`

Fase 3 – Prologo lato chiamato

- Eventuale allocazione del call-frame sullo stack (aggiornare `sp`)
 - Salvataggio di `x8 (fp)`, solo se utilizzato all'interno della procedura
 - Salvataggio di `x9 e x18-x27 (s1-s11)` se utilizzati all'interno della procedura (il chiamante si aspetta di trovarli intatti)
- Eventuale inizializzazione di `fp`: punta al nuovo call-frame

```
sommaArray:
    li t0, 0          # t0 = somma
    li t1, 0          # t1 = contatore indice i
loop:
    bge t1, a1, fine  # i >= lunghezza, esci dal loop
    slli t2, t1, 2    # offset
    add t3, a0, t2    # indirizzo elemento i
    lw t4, 0(t3)      # Carica v[i]
    add t0, t0, t4     # Aggiorna la somma
    addi t1, t1, 1    # i++
    j loop            # Ripeti il ciclo

fine:
    mv a0, t0          # Restituisci il risultato in a0
    jr ra              # Ritorna

# Funzione main
_start:
    la a0, array       # Indirizzo dell'array in a0
    la a1, length      # Lunghezza dell'array in a1
    lw a1, 0(a1)
    jal ra, sommaArray

    li a7, 93          # syscall exit (codice in a0)
    ecall              # Termina il programma
```

Le fasi di una invocazione di procedura foglia



Fase 4 – Corpo della procedura

- Istruzioni che implementano il corpo della procedura

Fase 5 – Epilogo lato chiamato

- Se deve essere restituito un valore dalla funzione
 - Tale valore viene posto in x10 (e x11) ovvero a0-a1
- I registri (se salvati) devono essere ripristinati
 - x9 e x18-x27 (s1-s11)
 - x8 (fp)
- Notare che sp deve solo essere aumentato di opportuno offset (lo stesso sottratto nella Fase 3)

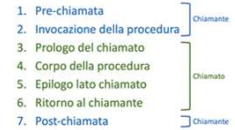
```
sommaArray:
    li t0, 0          # t0 = somma
    li t1, 0          # t1 = contatore indice i
loop:
    bge t1, a1, fine  # i >= lunghezza, esci dal loop
    slli t2, t1, 2     # offset
    add t3, a0, t2     # indirizzo elemento i
    lw t4, 0(t3)       # Carica v[i]
    add t0, t0, t4     # Aggiorna la somma
    addi t1, t1, 1     # i++
    j loop            # Ripeti il ciclo

fine:
    mv a0, t0          # Restituisci il risultato in a0
    jr ra              # Ritorna

# Funzione main
_start:
    la a0, array       # Indirizzo dell'array in a0
    la a1, length      # Lunghezza dell'array in a1
    lw a1, 0(a1)
    jal ra, sommaArray

    li a7, 93          # syscall exit (codice in a0)
    ecall              # Termina il programma
```

Le fasi di una invocazione di procedura foglia



Fase 6 – Ritorno al chiamante

- Istruzione `jalr x0, 0(x1)` (o pseudo-istruzione `jr ra`)

Fase 7 – Post-chiamata lato chiamante

- Eventuale uso del risultato della funzione (in `x10` e `x11`, cioè `a0-a1`)
- Ripristino dei valori `x5-x7` e `x28-x31` (`t0-t6`), `x10-x17` (`a0-a7`) vecchi, eventualmente salvati

```
sommaArray:
    li t0, 0          # t0 = somma
    li t1, 0          # t1 = contatore indice i
loop:
    bge t1, a1, fine  # i >= lunghezza, esci dal loop
    slli t2, t1, 2     # offset
    add t3, a0, t2     # indirizzo elemento i
    lw t4, 0(t3)       # Carica v[i]
    add t0, t0, t4     # Aggiorna la somma
    addi t1, t1, 1     # i++
    j loop            # Ripeti il ciclo

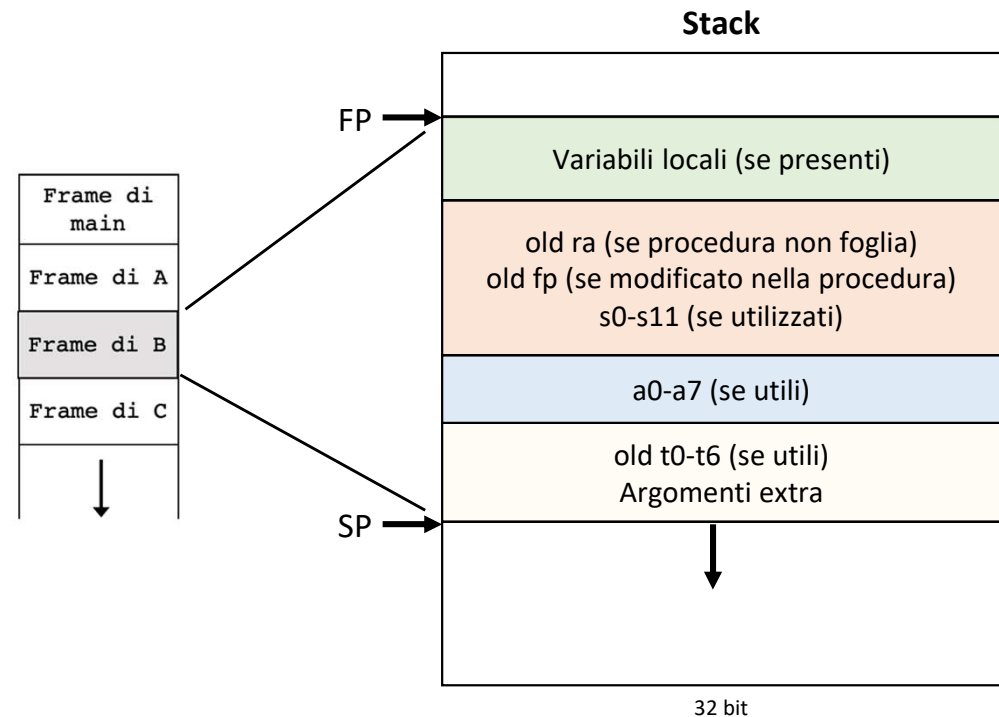
fine:
    mv a0, t0          # Restituisci il risultato in a0
    jr ra              # Ritorna

# Funzione main
_start:
    la a0, array       # Indirizzo dell'array in a0
    la a1, length      # Lunghezza dell'array in a1
    lw a1, 0(a1)
    jal ra, sommaArray

    li a7, 93          # syscall exit (codice in a0)
    ecall              # Termina il programma
```

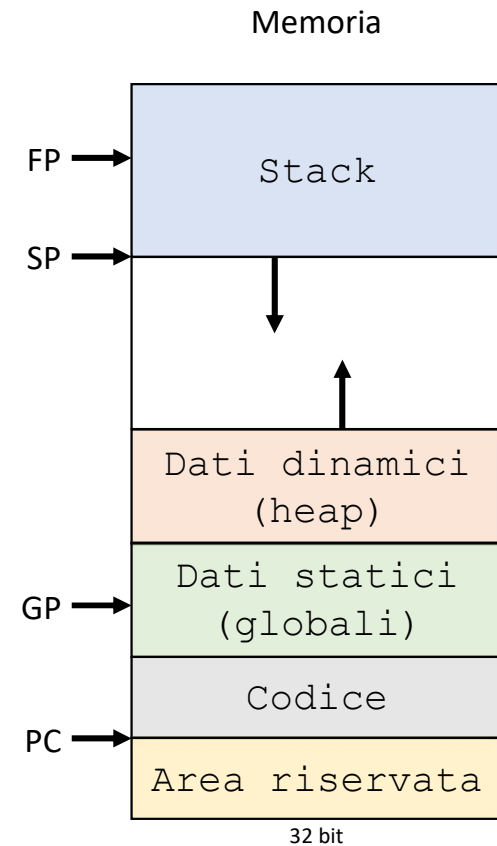

Record di attivazione: struttura

- Se utilizzato, il registro fp (frame pointer) viene inizializzato al valore di sp all'inizio della chiamata
- fp consente di avere un riferimento alle variabili locali che non muta con l'esecuzione della procedura
- Se lo stack non contiene variabili locali alla procedura, il compilatore risparmia tempo di esecuzione evitando di impostare e ripristinare il frame.



Organizzazione della memoria

- In memoria, oltre allo stack, trovano posto
 - Dati allocati dinamicamente (ad esempio, attraverso la malloc() del C)
 - Dati statici e variabili globali
 - Codice del programma
- Lo stack e i dati dinamici crescono in direzioni differenti, in modo da ottimizzarne la gestione



Esempio: area di un triangolo

```
int moltiplicazione(int a, int b) {
    int rst=a;
    for(int i=1;i<b;i++)
        rst = rst+a;
    return rst;
}

int area(int base, int altezza) {
    int rst = moltiplicazione(base, altezza)/2;
    return rst;
}
...
printf("Area = %d\n",area(20,23));
...
```

Si supponga di non avere una operazione di moltiplicazione

1. Pre-chiamata
 2. Invocazione della procedura
 3. Prologo del chiamato
 4. Corpo della procedura
 5. Epilogo lato chiamato
 6. Ritorno al chiamante
 7. Post-chiamata
- Chiamante
- Chiamato
- Chiamante

Invocazione della funzione area con parametri base=20 ,altezza=23

```
_start:
    li a0, 20 # salvo la base in a0
    li a1, 23 # salvo l'altezza in a1

# chiama area(base,altezza)
jal ra, area      # altezza in a0, base in a1
add t0, a0, zero  # salva il risultato in t0

# stampa messaggio per il risultato
la a0, visris     # pseudo-istruzione
addi a7, zero, 4
ecall

# stampa il risultato
add a0, t0, zero
addi a7, zero, 1
ecall

# stampa \n
la a0, RTN        # pseudo-istruzione
addi a7, zero, 4
ecall

# exit
addi a7, zero, 10
ecall
```



Il codice può essere ottimizzato

Esempio: area di un triangolo

```
int moltiplicazione(int a, int b) {
    int rst=a;
    for(int i=1;i<b;i++)
        rst = rst+a;
    return rst;
}

int area(int base, int altezza) {
    int rst = moltiplicazione(base, altezza)/2;
    return rst;
}

...
printf("Area = %d\n", area(20,23));
...
```

Si supponga di non avere una operazione di moltiplicazione

1. Pre-chiamata
 2. Invocazione della procedura
 3. Prologo del chiamato
 4. Corpo della procedura
 5. Epilogo lato chiamato
 6. Ritorno al chiamante
 7. Post-chiamata
- Chiamante
- Chiamato
- Chiamante

Funzione area con invocazione della funzione moltiplicazione

```
area:
# crea il call frame sullo stack (12 byte=ra+fp+rst)
# lo stack cresce verso il basso
    addi sp, sp, -12    # allocazione del call frame nello stack
    sw    ra, 4(sp)     # salvataggio dell'indirizzo di ritorno
    sw    fp, 0(sp)     # salvataggio del precedente frame pointer
    addi fp, sp, 8      # aggiornamento del frame pointer

# calcolo dell'area
    jal ra, moltiplicazione
    sw a0, 0(fp)        # salva il risultato della moltiplicazione
                                # nella variabile locale rst

    srai a0,a0,1

# uscita dalla funzione
    lw    fp, 0(sp)     # recupera il frame pointer
    lw    ra, 4(sp)     # recupera l'indirizzo di ritorno
    addi sp, sp, 12     # elimina il call frame dallo stack
    jr    ra            # ritorna al chiamante, pseudo-istruzione
```



Il codice può essere ottimizzato

Esempio: area di un triangolo

```
int moltiplicazione(int a, int b) {
    int rst=a;
    for(int i=1;i<b;i++)
        rst = rst+a;
    return rst;
}

int area(int base, int altezza) {
    int rst = moltiplicazione(base, altezza)/2;
    return rst;
}

...
printf("Area = %d\n",area(20,23));
...
```

Si supponga di non avere una operazione di moltiplicazione

1. Pre-chiamata
 2. Invocazione della procedura
 3. Prologo del chiamato
 4. Corpo della procedura
 5. Epilogo lato chiamato
 6. Ritorno al chiamante
 7. Post-chiamata
- Chiamante
- Chiamato
- Chiamante

Funzione moltiplicazione

```
moltiplicazione:
    addi sp, sp, -8      # allocazione del call frame nello stack
    sw ra, 4(sp)      # salvataggio dell'indirizzo di ritorno
    sw fp, 0(sp)         # salvataggio del precedente frame pointer
    addi fp, sp, 4       # aggiornamento del frame pointer
    sw a0, 0(fp)         # salvataggio variabile locale

    lw t2, 0(fp)
    li t0, 1
for:
    bge t0,a1,endfor
    add t2,a0,t2
    addi t0,t0,1
    j for                # pseudo-istruzione
endfor:
    lw fp, 0(sp)         # recupera il frame pointer
    lw ra, 4(sp)      # recupera l'indirizzo di ritorno
    addi sp, sp, 8       # elimina il call frame dallo stack
    add a0,t2,zero       # valore di ritorno in a0
    jr ra                # ritorna al chiamante, pseudo-istruzione
```



Il codice può essere ottimizzato

Esempio: area di un triangolo

```
int moltiplicazione(int a, int b) {
    int rst=a;
    for(int i=1;i<b;i++)
        rst = rst+a;
    return rst;
}

int area(int base, int altezza) {
    int rst = moltiplicazione(base, altezza)/2;
    return rst;
}

...
printf("Area = %d\n",area(20,23));
...
```

Si supponga di non avere una operazione di moltiplicazione

1. Pre-chiamata
 2. Invocazione della procedura
 3. Prologo del chiamato
 4. Corpo della procedura
 5. Epilogo lato chiamato
 6. Ritorno al chiamante
 7. Post-chiamata
- Chiamante

Chiamato

Chiamante

 Il codice può essere ottimizzato

RISC-V Instruction Set

```
moltiplicazione:
addi sp, sp, -8 # allocazione del call frame nello stack
sw ra, 4(sp) # salvataggio dell'indirizzo di ritorno
sw fp, 0(sp) # salvataggio del precedente frame pointer
addi fp, sp, 4 # aggiornamento del frame pointer
sw a0, 0(fp) # salvataggio variabile locale
lw t2, 0(fp)
li t0, 1
for:
bge t0, a1, endfor
add t2, a0, t2
addi t0, t0, 1
j for # pseudo-istruzione
endfor:
lw fp, 0(sp) # recupera il frame pointer
lw ra, 4(sp) # recupera l'indirizzo di ritorno
addi sp, sp, 8 # elimina il call frame dallo stack
add a0, t2, zero # valore di ritorno in a0
jr ra # ritorna al chiamante, pseudo-istruzione

area:
# crea il call frame sullo stack (12 byte=ra+fp+rst)
addi sp, sp, -12 # allocazione del call frame nello stack
sw ra, 4(sp) # salvataggio dell'indirizzo di ritorno
sw fp, 0(sp) # salvataggio del precedente frame pointer
addi fp, sp, 8 # aggiornamento del frame pointer
# calcolo dell'area
jal ra, moltiplicazione
sw a0, 0(fp) # salva il risultato della moltiplicazione in rst
srai a0, a0, 1
# uscita dalla funzione
lw fp, 0(sp) # recupera il frame pointer
lw ra, 4(sp) # recupera l'indirizzo di ritorno
addi sp, sp, 12 # elimina il call frame dallo stack
jr ra # ritorna al chiamante, pseudo-istruzione

start:
li a0, 20 # salvo la base in a0
li a1, 23 # salvo l'altezza in a1
# chiama area(base, altezza)
jal ra, area # altezza in a0, base in a1
add t0, a0, zero # salva il risultato in t0
# stampa messaggio per il risultato
la a0, visris # pseudo-istruzione
addi a7, zero, 4
ecall
# stampa il risultato
add a0, t0, zero
addi a7, zero, 1
ecall
# stampa \n
la a0, RTN # pseudo-istruzione
addi a7, zero, 4
ecall
# exit
addi a7, zero, 10
ecall
```

46

Un esempio

```

_start:
0x00400000  li a0, 20 # salvo la base in a0
0x00400004  li a1, 23 # salvo l'altezza in a1

# chiama triangolo(base,altezza)
0x00400008  jal ra, area # altezza in a0, base in a1
0x0040000C  add t0, a0, zero # salva il risultato in t0
...

area:
# crea il call frame sullo stack (12 byte=ra+fp+rst)
# lo stack cresce verso il basso
0x00400010  addi sp, sp, -12 # allocazione del call frame nello stack
0x00400014  sw ra, 4(sp) # salvataggio dell'indirizzo di ritorno
0x00400018  sw fp, 0(sp) # salvataggio del precedente frame pointer
0x0040001C  addi fp, sp, 8 # aggiornamento del frame pointer

# calcolo dell'area
0x00400020  jal ra, moltiplicazione
0x00400024  sw a0, 0(fp) #salva il risultato della moltiplicazione
                                #nella variabile locale rst
0x00400028  srai a0,a0,1

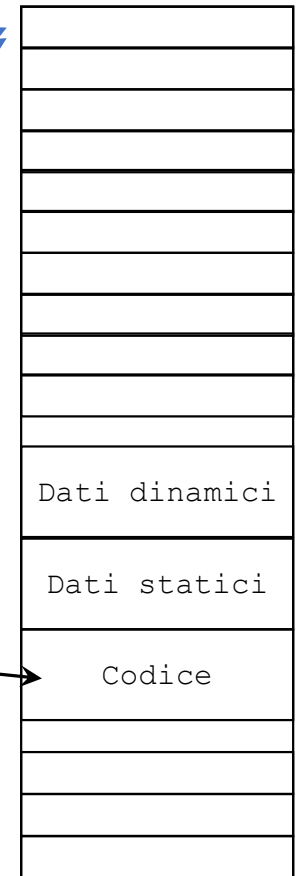
# uscita dalla funzione
0x0040002C  lw fp, 0(sp) # recupera il frame pointer
0x00400030  lw ra, 4(sp) # recupera l'indirizzo di ritorno
0x00400034  addi sp, sp, 12 # elimina il call frame dallo stack
0x00400038  jr ra # ritorna al chiamante
...

```

RISC-V Instruction Set

a0	20
a1	23
t0	
s0/fp	
sp	
ra	-

Memoria



32 bit

- Prossima istruzione da eseguire
- Istruzioni eseguite

Un esempio

```

_start:
0x00400000  li a0, 20 # salvo la base in a0
0x00400004  li a1, 23 # salvo l'altezza in a1

# chiama triangolo(base,altezza)
0x00400008  jal ra, area      # altezza in a0, base in a1
0x0040000C  add t0, a0, zero   # salva il risultato in t0
...
area:
# crea il call frame sullo stack (12 byte=ra+fp+rst)
# lo stack cresce verso il basso
0x00400010  addi sp, sp, -12    # allocazione del call frame nello stack
0x00400014  sw  ra, 4(sp)       # salvataggio dell'indirizzo di ritorno
0x00400018  sw  fp, 0(sp)       # salvataggio del precedente frame pointer
0x0040001C  addi fp, sp, 8      # aggiornamento del frame pointer

# calcolo dell'area
0x00400020  jal ra, moltiplicazione
0x00400024  sw a0, 0(fp)        #salva il risultato della moltiplicazione
                                #nella variabile locale rst
0x00400028  srai a0,a0,1

# uscita dalla funzione
0x0040002C  lw  fp, 0(sp)       # recupera il frame pointer
0x00400030  lw  ra, 4(sp)       # recupera l'indirizzo di ritorno
0x00400034  addi sp, sp, 12     # elimina il call frame dallo stack
0x00400038  jr  ra              # ritorna al chiamante
...

```

RISC-V Instruction Set

a0	20
a1	23
t0	
s0/fp	
sp	
ra	0x0040000C



■ Prossima istruzione da eseguire

■ Istruzioni eseguite

Un esempio

```

_start:
0x00400000  li a0, 20 # salvo la base in a0
0x00400004  li a1, 23 # salvo l'altezza in a1

# chiama triangolo(base,altezza)
0x00400008  jal ra, area      # altezza in a0, base in a1
0x0040000C  add t0, a0, zero   # salva il risultato in t0
...

area:
# crea il call frame sullo stack (12 byte=ra+fp+rst)
# lo stack cresce verso il basso
0x00400010  addi sp, sp, -12    # allocazione del call frame nello stack
0x00400014  sw  ra, 4(sp)       # salvataggio dell'indirizzo di ritorno
0x00400018  sw  fp, 0(sp)       # salvataggio del precedente frame pointer
0x0040001C  addi fp, sp, 8      # aggiornamento del frame pointer

# calcolo dell'area
0x00400020  jal ra, moltiplicazione
0x00400024  sw a0, 0(fp)        #salva il risultato della moltiplicazione
                                #nella variabile locale rst
0x00400028  srai a0,a0,1

# uscita dalla funzione
0x0040002C  lw  fp, 0(sp)       # recupera il frame pointer
0x00400030  lw  ra, 4(sp)       # recupera l'indirizzo di ritorno
0x00400034  addi sp, sp, 12     # elimina il call frame dallo stack
0x00400038  jr  ra             # ritorna al chiamante
...

```

RISC-V Instruction Set

a0	20
a1	23
t0	
s0/fp	
sp	
ra	0x0040000C



■ Prossima istruzione da eseguire
 ■ Istruzioni eseguite

Un esempio

```
_start:
0x00400000  li a0, 20 # salvo la base in a0
0x00400004  li a1, 23 # salvo l'altezza in a1

# chiama triangolo(base,altezza)
0x00400008  jal ra, area      # altezza in a0, base in a1
0x0040000C  add t0, a0, zero   # salva il risultato in t0
...

area:
# crea il call frame sullo stack (12 byte=ra+fp+rst)
# lo stack cresce verso il basso
0x00400010  addi sp, sp, -12    # allocazione del call frame nello stack
0x00400014  sw ra, 4(sp)       # salvataggio dell'indirizzo di ritorno
0x00400018  sw fp, 0(sp)       # salvataggio del precedente frame pointer
0x0040001C  addi fp, sp, 8     # aggiornamento del frame pointer

# calcolo dell'area
0x00400020  jal ra, moltiplicazione
0x00400024  sw a0, 0(fp)       #salva il risultato della moltiplicazione
                                #nella variabile locale rst
0x00400028  srai a0,a0,1

# uscita dalla funzione
0x0040002C  lw fp, 0(sp)       # recupera il frame pointer
0x00400030  lw ra, 4(sp)       # recupera l'indirizzo di ritorno
0x00400034  addi sp, sp, 12    # elimina il call frame dallo stack
0x00400038  jr ra             # ritorna al chiamante
...
RISC-V Instruction Set
```

a0	20
a1	23
t0	
s0/fp	
sp	
ra	0x00400024



■ Prossima istruzione da eseguire
■ Istruzioni eseguite

Un esempio

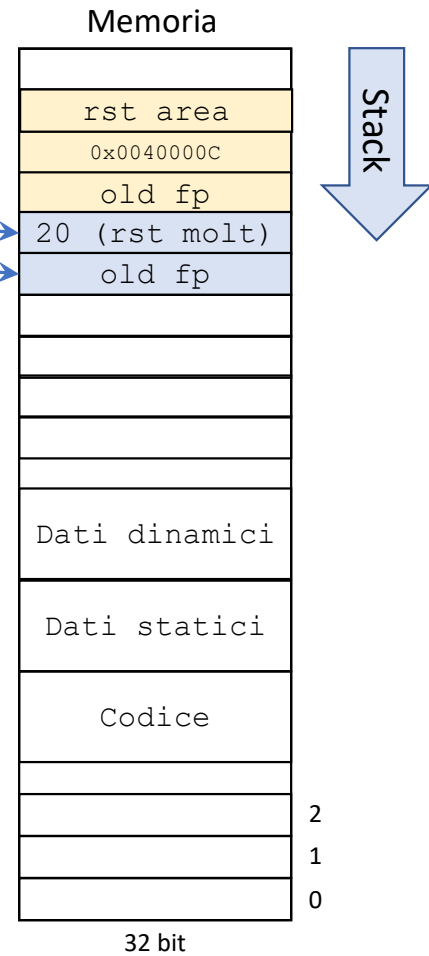
moltiplicazione:

```
0x00400038  addi sp, sp, -8      # allocazione del call frame nello stack
0x0040003C  sw  fp, 0(sp)        # salvataggio del precedente frame pointer
0x00400040  addi fp, sp, 4       # aggiornamento del frame pointer
0x00400044  sw a0, 0(fp)         # salvataggio variabile locale
```

```
0x00400048    lw t2, 0(fp)
0x0040004C    li t0, 1
for:
0x00400050    bge t0,a1,endifor
0x00400054    add t2,a0,t2
0x00400058    addi t0,t0,1
0x0040005C    j for
```

```
endfor:
0x00400060  lw fp, 0(sp)           # recupera il frame pointer
0x00400064  addi sp, sp, 8         # elimina il call frame dallo stack
0x00400068  add a0,t2,zero        # valore di ritorno in a0
0x0040006C  jr   ra               # ritorna al chiamante
```

a0	20
a1	23
t0	
t2	
s0/fp	
sp	
ra	0x00400024



Un esempio

moltiplicazione:

```
0x00400038  addi sp, sp, -8      # allocazione del call frame nello stack
0x0040003C  sw   fp, 0(sp)     # salvataggio del precedente frame pointer
0x00400040  addi fp, sp, 4      # aggiornamento del frame pointer
0x00400044  sw   a0, 0(fp)     # salvataggio variabile locale
```

```
0x00400048  lw   t2, 0(fp)
0x0040004C  li   t0, 1
for:
0x00400050  bge t0,a1,endifor
0x00400054  add  t2,a0,t2
0x00400058  addi t0,t0,1
0x0040005C  j    for
```

```
endifor:
0x00400060  lw   fp, 0(sp)      # recupera il frame pointer
0x00400064  addi sp, sp, 8      # elimina il call frame dallo stack
0x00400068  add  a0,t2,zero      # valore di ritorno in a0
0x0040006C  jr    ra           # ritorna al chiamante
```

Al termine del ciclo for...

a0	20
a1	23
t0	23
t2	460
s0/fp	
sp	
ra	0x00400024



Un esempio

moltiplicazione:

```

0x00400038  addi sp, sp, -8      # allocazione del call frame nello stack
0x0040003C  sw  fp, 0(sp)      # salvataggio del precedente frame pointer
0x00400040  addi fp, sp, 4      # aggiornamento del frame pointer
0x00400044  sw a0, 0(fp)      # salvataggio variabile locale

0x00400048  lw t2, 0(fp)
0x0040004C  li t0, 1
for:
0x00400050  bge t0,a1,endifor
0x00400054  add t2,a0,t2
0x00400058  addi t0,t0,1
0x0040005C  j for
endifor:
0x00400060  lw fp, 0(sp)      # recupera il frame pointer
0x00400064  addi sp, sp, 8      # elimina il call frame dallo stack
0x00400068  add a0,t2,zero     # valore di ritorno in a0
0x0040006C  jr  ra            # ritorna al chiamante
    
```

a0	460
a1	23
t0	23
t2	460
s0/fp	
sp	
ra	0x00400024



Un esempio

[illegible]

moltiplicazione:

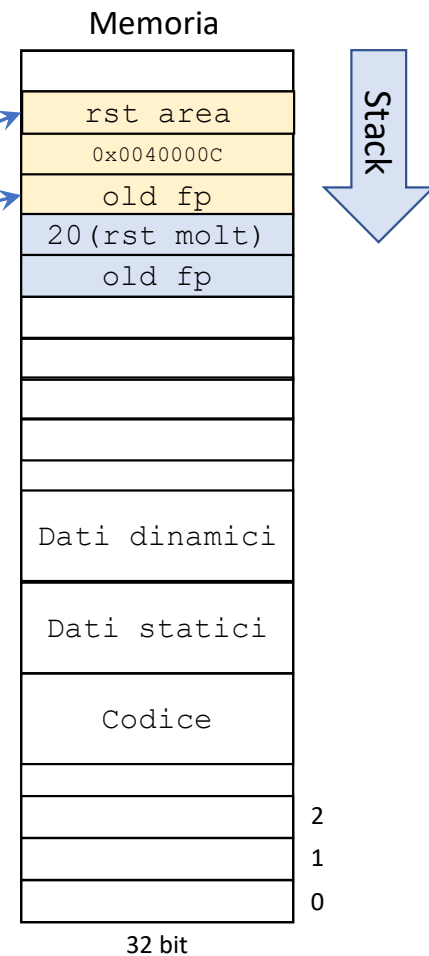
```

0x00400038    addi sp, sp, -8           # allocazione del call frame nello stack
0x0040003C    sw     fp, 0(sp)         # salvataggio del precedente frame pointer
0x00400040    addi   fp, sp, 4         # aggiornamento del frame pointer
0x00400044    sw     a0, 0(fp)         # salvataggio variabile locale

0x00400048    lw     t2, 0(fp)
0x0040004C    li     t0, 1
for:
0x00400050    bge    t0,a1,endfor
0x00400054    add    t2,a0,t2
0x00400058    addi   t0,t0,1
0x0040005C    j      for
endfor:
0x00400060    lw     fp, 0(sp)         # recupera il frame pointer
0x00400064    addi   sp, sp, 8         # elimina il call frame dallo stack
0x00400068    add    a0,t2,zero        # valore di ritorno in a0
0x0040006C    jr     ra                # ritorna al chiamante

```

a0	460
a1	23
t0	23
t2	460
s0/fp	
sp	
ra	0x00400024



Un esempio

```

_start:
0x00400000  li a0, 20 # salvo la base in a0
0x00400004  li a1, 23 # salvo l'altezza in a1

# chiama triangolo(base,altezza)
0x00400008  jal ra, area      # altezza in a0, base in a1
0x0040000C  add t0, a0, zero   # salva il risultato in t0
...

area:
# crea il call frame sullo stack (12 byte=ra+fp+rst)
# lo stack cresce verso il basso
0x00400010  addi sp, sp, -12    # allocazione del call frame nello stack
0x00400014  sw ra, 4(sp)        # salvataggio dell'indirizzo di ritorno
0x00400018  sw fp, 0(sp)        # salvataggio del precedente frame pointer
0x0040001C  addi fp, sp, 8      # aggiornamento del frame pointer

# calcolo dell'area
0x00400020  jal ra, moltiplicazione
0x00400024  sw a0, 0(fp)        #salva il risultato della moltiplicazione
                                #nella variabile locale rst
0x00400028  srai a0,a0,1        #divide per due

# uscita dalla funzione
0x0040002C  lw fp, 0(sp)        # recupera il frame pointer
0x00400030  lw ra, 4(sp)        # recupera l'indirizzo di ritorno
0x00400034  addi sp, sp, 12     # elimina il call frame dallo stack
0x00400038  jr ra              # ritorna al chiamante
...

```

a0	230
a1	23
t0	23
t2	460
s0/fp	
sp	
ra	0x00400024



■ Prossima istruzione da eseguire
■ Istruzioni eseguite

Un esempio

```

_start:
0x00400000  li a0, 20 # salvo la base in a0
0x00400004  li a1, 23 # salvo l'altezza in a1

# chiama triangolo(base,altezza)
0x00400008  jal ra, area      # altezza in a0, base in a1
0x0040000C  add t0, a0, zero   # salva il risultato in t0
...

area:
# crea il call frame sullo stack (12 byte=ra+fp+rst)
# lo stack cresce verso il basso
0x00400010  addi sp, sp, -12    # allocazione del call frame nello stack
0x00400014  sw ra, 4(sp)       # salvataggio dell'indirizzo di ritorno
0x00400018  sw fp, 0(sp)       # salvataggio del precedente frame pointer
0x0040001C  addi fp, sp, 8      # aggiornamento del frame pointer

# calcolo dell'area
0x00400020  jal ra, moltiplicazione
0x00400024  sw a0, 0(fp)       #salva il risultato della moltiplicazione
                                #nella variabile locale rst
0x00400028  srai a0,a0,1       #divide per due

# uscita dalla funzione
0x0040002C  lw fp, 0(sp)       # recupera il frame pointer
0x00400030  lw ra, 4(sp)       # recupera l'indirizzo di ritorno
0x00400034  addi sp, sp, 12    # elimina il call frame dallo stack
0x00400038  jr ra              # ritorna al chiamante
...

```

a0	230
a1	23
t0	23
t2	460
s0/fp	
sp	
ra	0x0040000C



■ Prossima istruzione da eseguire
■ Istruzioni eseguite

Un esempio

```
_start:
0x00400000  li a0, 20 # salvo la base in a0
0x00400004  li a1, 23 # salvo l'altezza in a1

# chiama triangolo(base,altezza)
0x00400008  jal ra, area      # altezza in a0, base in a1
0x0040000C  add t0, a0, zero   # salva il risultato in t0
...

area:
# crea il call frame sullo stack (12 byte=ra+fp+rst)
# lo stack cresce verso il basso
0x00400010  addi sp, sp, -12    # allocazione del call frame nello stack
0x00400014  sw ra, 4(sp)       # salvataggio dell'indirizzo di ritorno
0x00400018  sw fp, 0(sp)       # salvataggio del precedente frame pointer
0x0040001C  addi fp, sp, 8     # aggiornamento del frame pointer

# calcolo dell'area
0x00400020  jal ra, moltiplicazione
0x00400024  sw a0, 0(fp)       #salva il risultato della moltiplicazione
                                #nella variabile locale rst
0x00400028  srai a0,a0,1       #divide per due

# uscita dalla funzione
0x0040002C  lw fp, 0(sp)       # recupera il frame pointer
0x00400030  lw ra, 4(sp)       # recupera l'indirizzo di ritorno
0x00400034  addi sp, sp, 12    # elimina il call frame dallo stack
0x00400038  jr ra              # ritorna al chiamante
...
```

a0	230
a1	23
t0	23
t2	460
s0/fp	
sp	
ra	0x0040000C



■ Prossima istruzione da eseguire

■ Istruzioni eseguite

Un esempio

```

_start:
0x00400000  li a0, 20 # salvo la base in a0
0x00400004  li a1, 23 # salvo l'altezza in a1

# chiama triangolo(base,altezza)
0x00400008  jal ra, area      # altezza in a0, base in a1
0x0040000C  add t0, a0, zero   # salva il risultato in t0
...

area:
# crea il call frame sullo stack (12 byte=ra+fp+rst)
# lo stack cresce verso il basso
0x00400010  addi sp, sp, -12    # allocazione del call frame nello stack
0x00400014  sw ra, 4(sp)       # salvataggio dell'indirizzo di ritorno
0x00400018  sw fp, 0(sp)       # salvataggio del precedente frame pointer
0x0040001C  addi fp, sp, 8      # aggiornamento del frame pointer

# calcolo dell'area
0x00400020  jal ra, moltiplicazione
0x00400024  sw a0, 0(fp)       #salva il risultato della moltiplicazione
                                #nella variabile locale rst
0x00400028  srai a0,a0,1       #divide per due

# uscita dalla funzione
0x0040002C  lw fp, 0(sp)       # recupera il frame pointer
0x00400030  lw ra, 4(sp)       # recupera l'indirizzo di ritorno
0x00400034  addi sp, sp, 12    # elimina il call frame dallo stack
0x00400038  jr ra              # ritorna al chiamante
...

```

a0	230
a1	23
t0	230
t2	460
s0/fp	
sp	
ra	0x0040000C



■ Prossima istruzione da eseguire
■ Istruzioni eseguite

Esempio: calcolo del fattoriale

```
int fact(int n) {  
    if (n==0)  
        return 1;  
    else return n*fact(n-1)  
}
```

```
fact:  
# crea il call frame sullo stack (12 byte)  
# lo stack cresce verso il basso  
    addi sp, sp, -12      # allocazione del call frame nello stack  
    sw    a0, 8(sp)       # salvataggio di n nel call frame  
    sw    ra, 4(sp)       # salvataggio dell'indirizzo di ritorno  
    sw    fp, 0(sp)       # salvataggio del precedente frame pointer  
    addi fp, sp, 8        # aggiornamento del frame pointer  
  
# calcolo del fattoriale  
    bne a0, zero, Ric     # test fine ricorsione n!=0  
    addi a0, zero, 1      # 0! = 1  
    j     Fine  
  
Ric:  
    addi a0, a0, -1       # chiamata ricorsiva per il calcolo di (n-1)!  
    jal fact              # a0 <- (n - 1) passaggio del parametro in a0 per fact(n-1)  
    lw    t0, 0(fp)       # chiama fact(n-1) → risultato in a0  
    mul   a0, a0, t0       # t0 <- n  
    mul   a0, a0, t0       # n! = (n-1)! x n  
  
# uscita dalla funzione  
Fine:  
    lw    fp, 0(sp)       # recupera il frame pointer  
    lw    ra, 4(sp)       # recupera l'indirizzo di ritorno  
    addi sp, sp, 12       # elimina il call frame dallo stack  
    jr    ra              # ritorna al chiamante
```

Esempio: calcolo del fattoriale

```

_start:
0x400000  li a0, 3 # salvo n in a0

0x400004  jal ra, fact # n in a0
0x400008  add t0, a0, zero # salva il risultato in t0

fact:
0x40000c  addi sp, sp, -12 # allocazione del call frame nello stack
0x400010  sw a0, 8(sp) # salvataggio di n nel call frame
0x400014  sw ra, 4(sp) # salvataggio dell'indirizzo di ritorno
0x400018  sw fp, 0(sp) # salvataggio del precedente frame pointer
0x40001c  addi fp, sp, 8 # aggiornamento del frame pointer

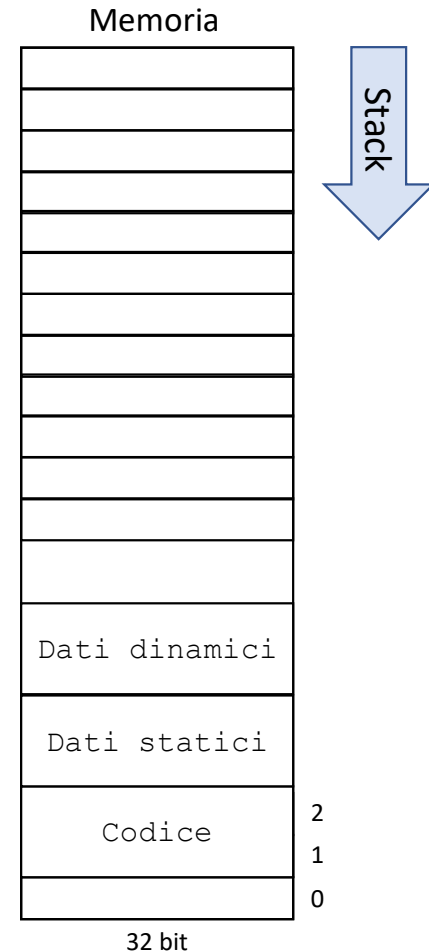
0x400020  bne a0, zero, Ric # test fine ricorsione n!=0
0x400024  addi a0, zero, 1 # 0! = 1
0x400028  j Fine

ric:
0x40002c  addi a0, a0, -1 # chiamata ricorsiva per il calcolo di (n-1)!
0x400030  jal fact # chiama fact(n-1) → risultato in a0
0x400034  lw t0, 0(fp) # t0 <- n
0x400038  mul a0, a0, t0 # n! = (n-1)! x n

fine:
0x40003c  lw fp, 0(sp) # recupera il frame pointer
0x400040  lw ra, 4(sp) # recupera l'indirizzo di ritorno
0x400044  addi sp, sp, 12 # elimina il call frame dallo stack
0x400048  jr ra # ritorna al chiamante
    
```

Prima dell'invocazione di
fact(3)

a0	3
t0	
s0/fp	
sp	
ra	



Esempio: calcolo del fattoriale

```

_start:
0x400000  li a0, 3 # salvo n in a0

0x400004  jal ra, fact      # n in a0
0x400008  add t0, a0, zero  # salva il risultato in t0

fact:
0x40000c  addi sp, sp, -12  # allocazione del call frame nello stack
0x400010  sw  a0, 8(sp)    # salvataggio di n nel call frame
0x400014  sw  ra, 4(sp)    # salvataggio dell'indirizzo di ritorno
0x400018  sw  fp, 0(sp)    # salvataggio del precedente frame pointer
0x40001c  addi fp, sp, 8   # aggiornamento del frame pointer

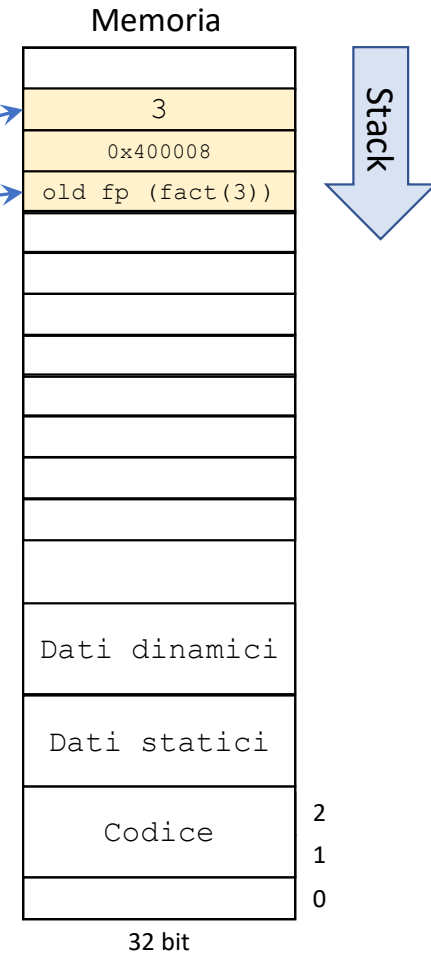
0x400020  bne a0, zero, Ric # test fine ricorsione n!=0
0x400024  addi a0, zero, 1  # 0! = 1
0x400028  j    Fine

ric:
0x40002c  addi a0, a0, -1   # chiamata ricorsiva per il calcolo di (n-1)!
0x400030  jal fact         # chiama fact(n-1) → risultato in a0
0x400034  lw  t0, 0(fp)    # t0 <- n
0x400038  mul a0, a0, t0    # n! = (n-1)! x n

fine:
0x40003c  lw  fp, 0(sp)    # recupera il frame pointer
0x400040  lw  ra, 4(sp)    # recupera l'indirizzo di ritorno
0x400044  addi sp, sp, 12  # elimina il call frame dallo stack
0x400048  jr  ra          # ritorna al chiamante
    
```

a0	3
t0	
s0/fp	
sp	
ra	0x400008

Dopo l'invocazione e della costruzione del record di attivazione di fact(3)



Esempio: calcolo del fattoriale

```
_start:
0x400000  li a0, 3 # salvo n in a0

0x400004  jal ra, fact      # n in a0
0x400008  add t0, a0, zero  # salva il risultato in t0

fact:
0x40000c  addi sp, sp, -12  # allocazione del call frame nello stack
0x400010  sw  a0, 8(sp)    # salvataggio di n nel call frame
0x400014  sw  ra, 4(sp)    # salvataggio dell'indirizzo di ritorno
0x400018  sw  fp, 0(sp)    # salvataggio del precedente frame pointer
0x40001c  addi fp, sp, 8   # aggiornamento del frame pointer

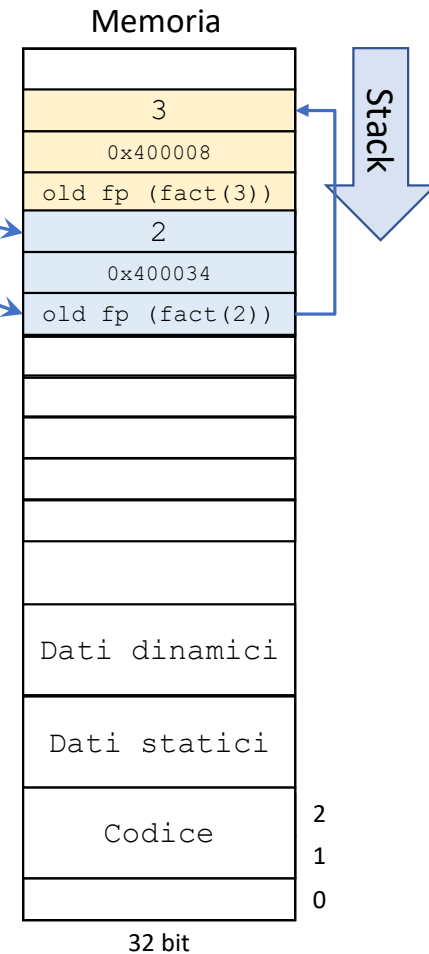
0x400020  bne a0, zero, Ric # test fine ricorsione n!=0
0x400024  addi a0, zero, 1  # 0! = 1
0x400028  j   Fine

ric:
0x40002c  addi a0, a0, -1   # chiamata ricorsiva per il calcolo di (n-1)!
0x400030  jal fact         # chiama fact(n-1) → risultato in a0
0x400034  lw  t0, 0(fp)    # t0 <- n
0x400038  mul a0, a0, t0   # n! = (n-1)! x n

fine:
0x40003c  lw  fp, 0(sp)    # recupera il frame pointer
0x400040  lw  ra, 4(sp)    # recupera l'indirizzo di ritorno
0x400044  addi sp, sp, 12  # elimina il call frame dallo stack
0x400048  jr  ra          # ritorna al chiamante
```

a0	2
t0	
s0/fp	
sp	
ra	0x400034

Dopo la chiamata ricorsiva
e la costruzione del record
di attivazione di fact(2)



Esempio: calcolo del fattoriale

```

_start:
0x400000  li a0, 3 # salvo n in a0

0x400004  jal ra, fact      # n in a0
0x400008  add t0, a0, zero  # salva il risultato in t0

fact:
0x40000c  addi sp, sp, -12  # allocazione del call frame nello stack
0x400010  sw  a0, 8(sp)    # salvataggio di n nel call frame
0x400014  sw  ra, 4(sp)    # salvataggio dell'indirizzo di ritorno
0x400018  sw  fp, 0(sp)    # salvataggio del precedente frame pointer
0x40001c  addi fp, sp, 8   # aggiornamento del frame pointer

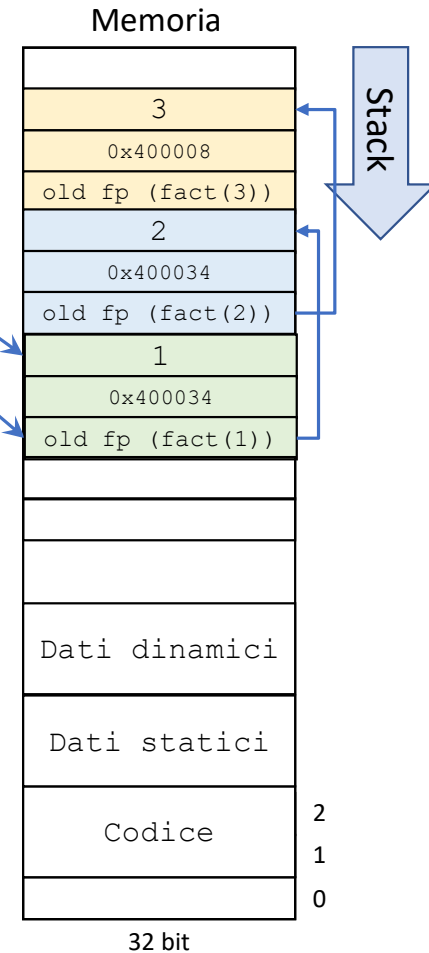
0x400020  bne a0, zero, Ric # test fine ricorsione n!=0
0x400024  addi a0, zero, 1  # 0! = 1
0x400028  j    Fine

ric:
0x40002c  addi a0, a0, -1   # chiamata ricorsiva per il calcolo di (n-1)!
0x400030  jal fact         # chiama fact(n-1) → risultato in a0
0x400034  lw  t0, 0(fp)     # t0 <- n
0x400038  mul a0, a0, t0    # n! = (n-1)! x n

fine:
0x40003c  lw  fp, 0(sp)    # recupera il frame pointer
0x400040  lw  ra, 4(sp)    # recupera l'indirizzo di ritorno
0x400044  addi sp, sp, 12  # elimina il call frame dallo stack
0x400048  jr  ra           # ritorna al chiamante
    
```

a0	1
t0	
s0/fp	
sp	
ra	0x400034

Dopo la chiamata ricorsiva
e la costruzione del record
di attivazione di fact(1)



Esempio: calcolo del fattoriale

```
_start:
0x400000  li a0, 3 # salvo n in a0

0x400004  jal ra, fact      # n in a0
0x400008  add t0, a0, zero  # salva il risultato in t0

fact:
0x40000c  addi sp, sp, -12  # allocazione del call frame nello stack
0x400010  sw  a0, 8(sp)    # salvataggio di n nel call frame
0x400014  sw  ra, 4(sp)    # salvataggio dell'indirizzo di ritorno
0x400018  sw  fp, 0(sp)    # salvataggio del precedente frame pointer
0x40001c  addi fp, sp, 8   # aggiornamento del frame pointer

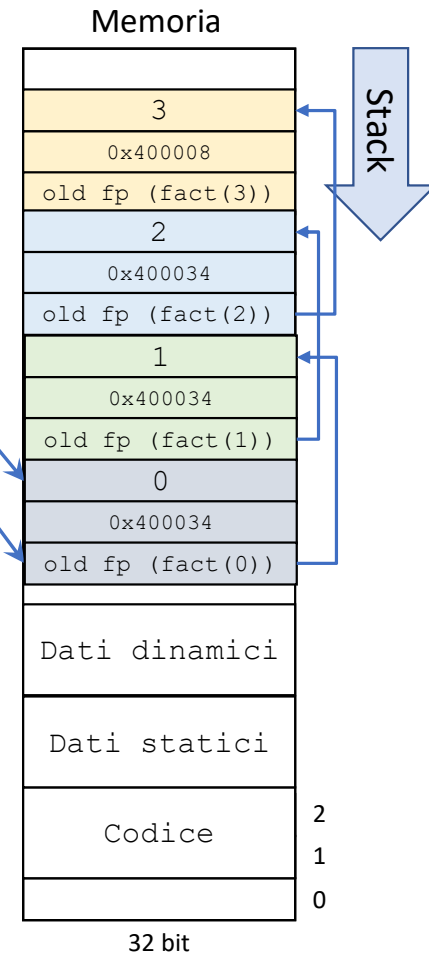
0x400020  bne a0, zero, Ric # test fine ricorsione n!=0
0x400024  addi a0, zero, 1  # 0! = 1
0x400028  j    Fine

ric:
0x40002c  addi a0, a0, -1   # chiamata ricorsiva per il calcolo di (n-1)!
0x400030  jal fact         # chiama fact(n-1) → risultato in a0
0x400034  lw  t0, 0(fp)    # t0 <- n
0x400038  mul a0, a0, t0   # n! = (n-1)! x n

fine:
0x40003c  lw  fp, 0(sp)    # recupera il frame pointer
0x400040  lw  ra, 4(sp)    # recupera l'indirizzo di ritorno
0x400044  addi sp, sp, 12  # elimina il call frame dallo stack
0x400048  jr  ra          # ritorna al chiamante
```

a0	0
t0	
s0/fp	
sp	
ra	0x400034

Dopo la chiamata ricorsiva
e la costruzione del record
di attivazione di fact(0)



Esempio: calcolo del fattoriale

```

_start:
0x400000  li a0, 3 # salvo n in a0

0x400004  jal ra, fact      # n in a0
0x400008  add t0, a0, zero  # salva il risultato in t0

fact:
0x40000c  addi sp, sp, -12  # allocazione del call frame nello stack
0x400010  sw  a0, 8(sp)     # salvataggio di n nel call frame
0x400014  sw  ra, 4(sp)     # salvataggio dell'indirizzo di ritorno
0x400018  sw  fp, 0(sp)     # salvataggio del precedente frame pointer
0x40001c  addi fp, sp, 8    # aggiornamento del frame pointer

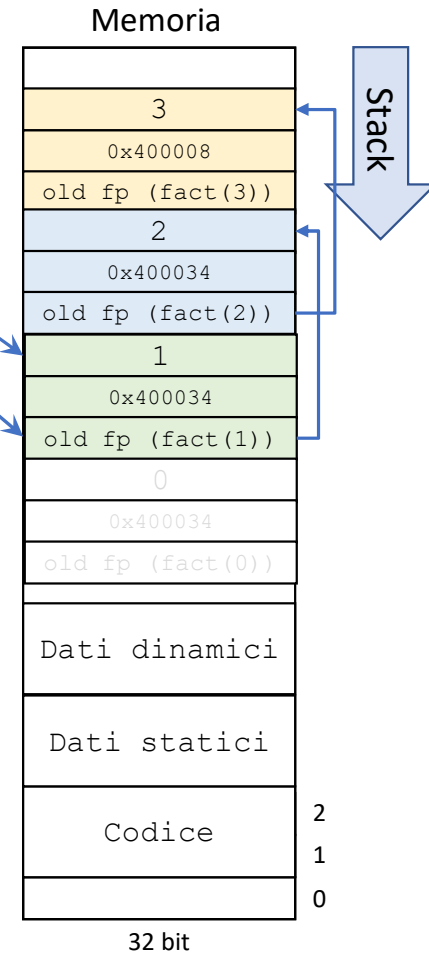
0x400020  bne a0, zero, Ric # test fine ricorsione n!=0
0x400024  addi a0, zero, 1  # 0! = 1
0x400028  j   Fine

ric:
0x40002c  addi a0, a0, -1   # chiamata ricorsiva per il calcolo di (n-1)!
0x400030  jal fact         # chiama fact(n-1) → risultato in a0
0x400034  lw  t0, 0(fp)     # t0 <- n
0x400038  mul a0, a0, t0    # n! = (n-1)! x n

fine:
0x40003c  lw  fp, 0(sp)     # recupera il frame pointer
0x400040  lw  ra, 4(sp)     # recupera l'indirizzo di ritorno
0x400044  addi sp, sp, 12   # elimina il call frame dallo stack
0x400048  jr  ra           # ritorna al chiamante
    
```

a0	1
t0	
s0/fp	
sp	
ra	0x400034

n=0 e quindi la funzione
ritorna 1, deallocando il
record di attivazione



Esempio: calcolo del fattoriale

```

_start:
0x400000  li a0, 3 # salvo n in a0

0x400004  jal ra, fact      # n in a0
0x400008  add t0, a0, zero  # salva il risultato in t0

fact:
0x40000c  addi sp, sp, -12  # allocazione del call frame nello stack
0x400010  sw  a0, 8(sp)     # salvataggio di n nel call frame
0x400014  sw  ra, 4(sp)     # salvataggio dell'indirizzo di ritorno
0x400018  sw  fp, 0(sp)     # salvataggio del precedente frame pointer
0x40001c  addi fp, sp, 8    # aggiornamento del frame pointer

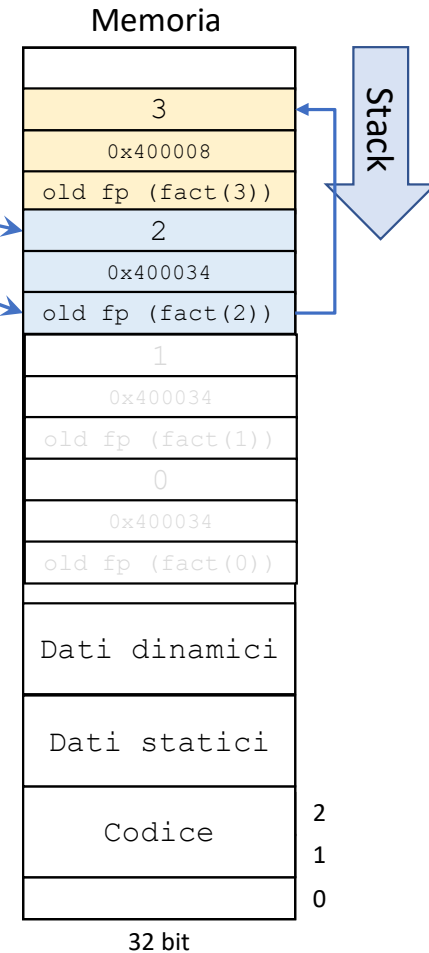
0x400020  bne a0, zero, Ric # test fine ricorsione n!=0
0x400024  addi a0, zero, 1  # 0! = 1
0x400028  j    Fine

ric:
0x40002c  addi a0, a0, -1   # chiamata ricorsiva per il calcolo di (n-1)!
0x400030  jal fact         # chiama fact(n-1) → risultato in a0
0x400034  lw  t0, 0(fp)     # t0 ← n
0x400038  mul a0, a0, t0    # n! = (n-1)! x n

fine:
0x40003c  lw  fp, 0(sp)     # recupera il frame pointer
0x400040  lw  ra, 4(sp)     # recupera l'indirizzo di ritorno
0x400044  addi sp, sp, 12   # elimina il call frame dallo stack
0x400048  jr  ra           # ritorna al chiamante
    
```

a0	1
t0	
s0/fp	
sp	
ra	0x400034

Viene calcolata la moltiplicazione 1x1 e viene ritornato il risultato, deallocando il record di attivazione



Esempio: calcolo del fattoriale

```

_start:
0x400000  li a0, 3 # salvo n in a0

0x400004  jal ra, fact      # n in a0
0x400008  add t0, a0, zero  # salva il risultato in t0

fact:
0x40000c  addi sp, sp, -12  # allocazione del call frame nello stack
0x400010  sw  a0, 8(sp)     # salvataggio di n nel call frame
0x400014  sw  ra, 4(sp)     # salvataggio dell'indirizzo di ritorno
0x400018  sw  fp, 0(sp)     # salvataggio del precedente frame pointer
0x40001c  addi fp, sp, 8    # aggiornamento del frame pointer

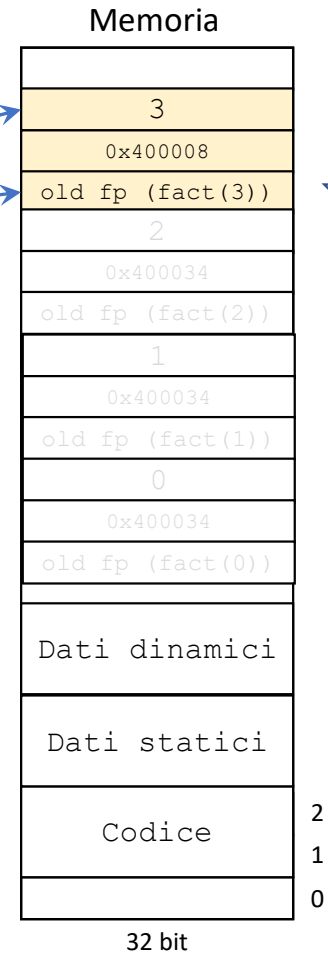
0x400020  bne a0, zero, Ric # test fine ricorsione n!=0
0x400024  addi a0, zero, 1  # 0! = 1
0x400028  j   Fine

ric:
0x40002c  addi a0, a0, -1   # chiamata ricorsiva per il calcolo di (n-1)!
0x400030  jal fact         # chiama fact(n-1) → risultato in a0
0x400034  lw  t0, 0(fp)     # t0 ← n
0x400038  mul a0, a0, t0    # n! = (n-1)! x n

fine:
0x40003c  lw  fp, 0(sp)     # recupera il frame pointer
0x400040  lw  ra, 4(sp)     # recupera l'indirizzo di ritorno
0x400044  addi sp, sp, 12   # elimina il call frame dallo stack
0x400048  jr  ra           # ritorna al chiamante
    
```

a0	2
t0	
s0/fp	
sp	
ra	0x400008

Viene calcolata la moltiplicazione 1x2 e viene ritornato il risultato, deallocando il record di attivazione



Esempio: calcolo del fattoriale

```

_start:
0x400000  li a0, 3 # salvo n in a0

0x400004  jal ra, fact      # n in a0
0x400008  add t0, a0, zero  # salva il risultato in t0

fact:
0x40000c  addi sp, sp, -12  # allocazione del call frame nello stack
0x400010  sw  a0, 8(sp)     # salvataggio di n nel call frame
0x400014  sw  ra, 4(sp)     # salvataggio dell'indirizzo di ritorno
0x400018  sw  fp, 0(sp)     # salvataggio del precedente frame pointer
0x40001c  addi fp, sp, 8    # aggiornamento del frame pointer

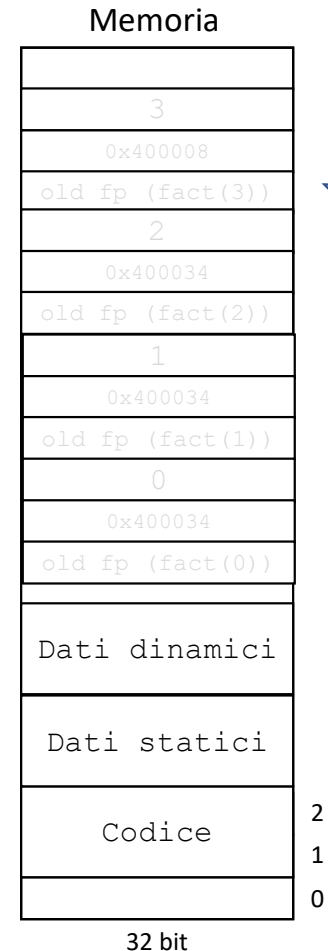
0x400020  bne a0, zero, Ric # test fine ricorsione n!=0
0x400024  addi a0, zero, 1  # 0! = 1
0x400028  j   Fine

ric:
0x40002c  addi a0, a0, -1   # chiamata ricorsiva per il calcolo di (n-1)!
0x400030  jal fact         # chiama fact(n-1) → risultato in a0
0x400034  lw  t0, 0(fp)     # t0 ← n
0x400038  mul a0, a0, t0    # n! = (n-1)! x n

fine:
0x40003c  lw  fp, 0(sp)     # recupera il frame pointer
0x400040  lw  ra, 4(sp)     # recupera l'indirizzo di ritorno
0x400044  addi sp, sp, 12   # elimina il call frame dallo stack
0x400048  jr  ra           # ritorna al chiamante
    
```

a0	6
t0	
s0/fp	
sp	
ra	

Viene calcolata la moltiplicazione 2x3 e viene ritornato il risultato al main, deallocando il record di attivazione



Convenzioni di chiamata: un esempio

- Data la funzione `moltiplica` che, dato un numero intero restituisca il risultato della sua moltiplicazione per 10 senza utilizzare istruzioni di moltiplicazione
- Scrivere una funzione `moltiplicaVettore` che
 - Dato un vettore di interi (word)
 - Restituisce il vettore sostituendo ogni elemento con il suo prodotto per 10

```
moltiplica:
    slli    t0, a0, 3      # t0 = a0 * 8
    slli    t1, a0, 1      # t1 = a0 * 2
    add     a0, t0, t1
    ret
```

Una possibile soluzione

a0: puntatore all'inizio del vettore
a1: numero di elementi nel vettore

salva il puntatore in t1 per il ciclo

carica l'elemento corrente in t0 e lo sposta
in a0 per la chiamata di moltiplica

salva il risultato al posto dell'elemento originale

passa al prossimo elemento

```
moltiplicaVettore:
    beq    a1, zero, fine
    mv     t1, a0
ciclo:
    lw     t0, 0(t1)
    mv     a0, t0
    jal    ra, moltiplica
    sw     a0, 0(t1)
    addi   t1, t1, 4
    addi   a1, a1, -1
    bnez   a1, ciclo
fine:
    ret
```

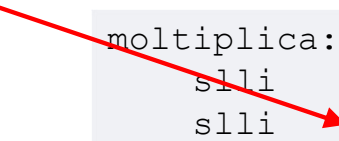
Una possibile soluzione

Problemi ? Molti!

- P1 - La funzione non è foglia ma il registro `ra` non viene salvato e ripristinato
- P2 - Convenzioni di chiamata non rispettate
 - Il registro `t1` viene modificato dall'invocazione di `moltiplica`
 - Il registro `a1` potrebbe essere modificato da `moltiplica`

```
moltiplicaVettore:
    beq     a1, zero, fine
    mv      t1, a0
ciclo:
    lw      t0, 0(t1)
    mv      a0, t0
    jal     ra, moltiplica
    sw      a0, 0(t1)
    addi    t1, t1, 4
    addi    a1, a1, -1
    bnez    a1, ciclo
fine:
    ret
```

```
moltiplica:
    slli    t0, a0, 3      # t0 = a0 * 8
    slli    t1, a0, 1      # t1 = a0 * 2
    add     a0, t0, t1
    ret
```

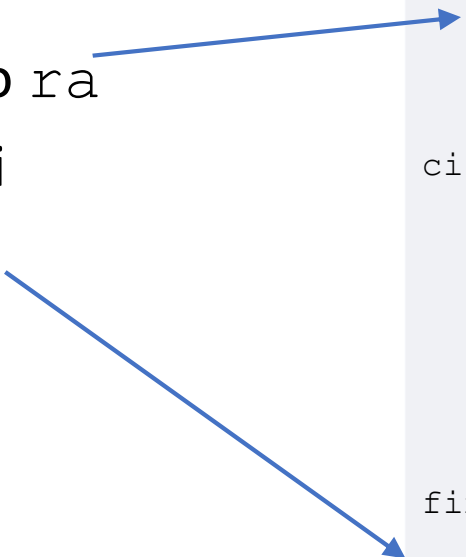


Una possibile soluzione

Risolviamo P1

- Creiamo un record di attivazione dove salviamo `ra`
- Ripristiniamo `ra` prima di tornare al chiamante e deallochiamo il record di attivazione

```
multiplicaVettore:
    addi    sp, sp, -4
    sw      ra, 0(sp)
    beq     a1, zero, fine
    mv      t1, a0
ciclo:
    lw      t0, 0(t1)
    mv      a0, t0
    jal     ra, multiplica
    sw      a0, 0(t1)
    addi    t1, t1, 4
    addi    a1, a1, -1
    bnez    a1, ciclo
fine:
    lw      ra, 0(sp)
    addi    sp, sp, 4
    ret
```



Una possibile soluzione

Risolviamo P2

- Salviamo in memoria `t1` e `a1` prima dell'invocazione di `moltiplica`
- Ripristiniamo dalla memoria il valore di `t1` e `a1` dopo l'invocazione di `moltiplica` ed il salvataggio

```
moltiplicaVettore:
    addi    sp, sp, -4
    sw      ra, 0(sp)
    beq     a1, zero, fine
    mv      t1, a0
ciclo:
    lw      t0, 0(t1)
    addi    sp, sp, -8
    sw      t1, 0(sp)
    sw      a1, 4(sp)
    mv      a0, t0
    jal     ra, moltiplica
    lw      t1, 0(sp)
    lw      a1, 4(sp)
    addi    sp, sp, 8
    sw      a0, 0(t1)
    addi    t1, t1, 4
    addi    a1, a1, -1
    bnez    a1, ciclo
fine:
    lw      ra, 0(sp)
    addi    sp, sp, 4
    ret
```

Una possibile soluzione

Risolviamo P2

- Salviamo in memoria t1 e a1 prima dell'invocazione di `moltiplica`
- Ripristiniamo dalla memoria il valore di t1 e a1 dopo l'invocazione di `moltiplica` ed il salvataggio

Inefficiente:

- Ad ogni iterazione vi è un salvataggio ed un caricamento in memoria!

```
moltiplicaVettore:
    addi    sp, sp, -4
    sw      ra, 0(sp)
    beq     a1, zero, fine
    mv      t1, a0
ciclo:
    lw      t0, 0(t1)
    addi    sp, sp, -8
    sw      t1, 0(sp)
    sw      a1, 4(sp)
    mv      a0, t0
    jal     ra, moltiplica
    lw      t1, 0(sp)
    lw      a1, 4(sp)
    addi    sp, sp, 8
    sw      a0, 0(t1)
    addi    t1, t1, 4
    addi    a1, a1, -1
    bnez    a1, ciclo
fine:
    lw      ra, 0(sp)
    addi    sp, sp, 4
    ret
```

Una possibile soluzione

Inefficiente:

- Ad ogni iterazione vi è un salvataggio ed un caricamento in memoria!

Soluzione

- Usiamo i registri s
 - I registri s non devono essere preservati dal chiamante, la funzione `moltiplicaVettore`
 - Ma `moltiplicaVettore` è anche una funzione chiamata (dal main, ad esempio)
 - Soluzione: salvare i registri s all'inizio della funzione e ripristinarli alla fine

```
moltiplicaVettore:
    addi    sp, sp, -12
    sw      s2, 8(sp)
    sw      s1, 4(sp)
    sw      ra, 0(sp)
    beq     a1, zero, fine
    mv      s1, a0
    mv      s2, a1
ciclo:
    lw      t0, 0(s1)
    mv      a0, t0
    jal     ra, moltiplica
    sw      a0, 0(s1)
    addi    s1, s1, 4
    addi    s2, s2, -1
    bnez    s2, ciclo
fine:
    lw      ra, 0(sp)
    lw      s1, 4(sp)
    lw      s2, 8(sp)
    addi    sp, sp, 12
    ret
```

Operandi immediati ampi

- Problema: è possibile caricare in un registro una costante a 32 bit? Supponiamo di voler caricare nel registro x5 il valore 0x12345678
- Soluzione
 - si introduce una nuova istruzione `lui` (load upper immediate, tipo U) che carica i 20 bit più significativi della costante nei bit da 12 a 31 di un registro
 - Con una operazione di or immediato si impostano i 12 bit meno significativi rimasti

```
lui x5, 0x12345
```

x5

.....

00010010 00110100 01010000 00000000

```
ori x5, x5, 0x678
```

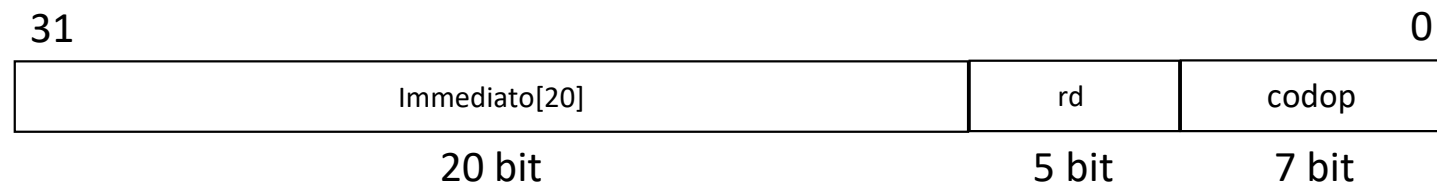
x5

.....

00010010 00110100 01010110 01111000

LUI e linguaggio macchina

- Viene introdotto un nuovo tipo: U

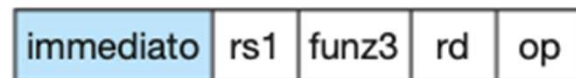


Operandi immediati ampi

- Potreste realizzare il caricamento con le istruzioni `lui` e `addi` (al posto di `ori`)?
 - Supponete di voler caricare nel registro x5 il valore `0x82345678`: succede qualcosa?
 - Supponete di voler caricare nel registro x5 il valore `0x12345878`: succede qualcosa?
 - Supponete di voler caricare nel registro x5 il valore `0x82345878`: succede qualcosa?

Modalità di indirizzamento

- Indirizzamento immediato

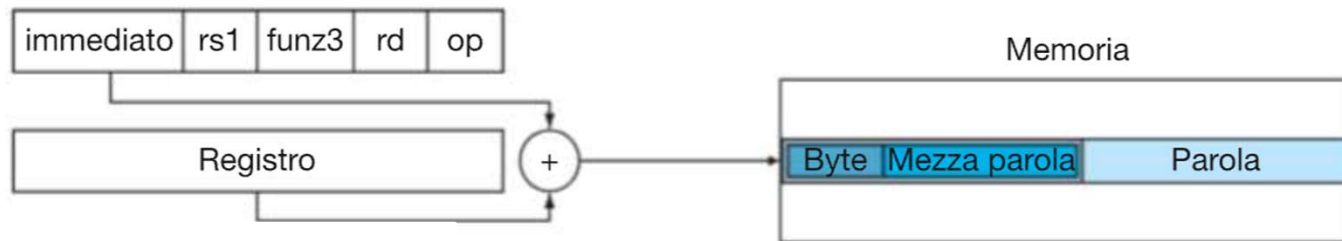


- Indirizzamento tramite registro



Modalità di indirizzamento

- Indirizzamento tramite base e spiazzamento



- Indirizzamento relativo al program counter



Salti con offset più grandi

- Come per le costanti, anche i salti possono essere eseguiti anche ad istruzioni più lontane
- RISC-V introduce la possibilità di salto in un intervallo pari a 2^{32}
- Nuova istruzione

```
auipc rd, offset
```

Add Upper Immediate PC
Tipo U in linguaggio macchina

- Inserisce nel registro `rd` l'indirizzo di `PC + (offset << 12)`

Esempio: `auipc x5, 0x12345`  `x5 = PC + 0x12345000`

Salti con offset più grandi

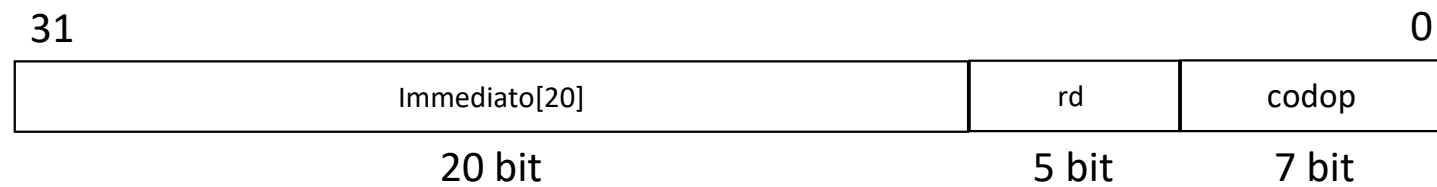
- Se usiamo `auipc` con i 20 bit più significativi dell'offset, allora possiamo aggiungere questa istruzione una istruzione che calcola
 - $PC = rd + \text{offset}[31..0]$
- Otteniamo come risultato un salto incondizionato con offset più esteso
- L'istruzione da considerare è `jalr`! Ricapitolando:

```
auipc rd, offset[31..12]  
jalr x0, offset[0..11](rd)
```

- Realizza un salto incondizionato a $PC + \text{offset}[31..0]$

AUIPC e linguaggio macchina

- Viene usato il tipo U



Riassunto dei formati delle istruzioni RISC-V

Nome (dimensione del campo)	Campi						Commenti
	7 bit	5 bit	5 bit	3 bit	5 bit	7 bit	
Tipo R	funz7	rs2	rs1	funz3	rd	codop	Istruzioni aritmetiche
Tipo I	Immediato[11:0]		rs1	funz3	rd	codop	Istruzioni di caricamento dalla memoria e aritmetica con costanti
Tipo S	immed[11:5]	rs2	rs1	funz3	immed[4:0]	codop	Istruzioni di trasferimento alla memoria (store)
Tipo SB	immed[12, 10:5]	rs2	rs1	funz3	immed[4:1,11]	codop	Istruzioni di salto condizionato
Tipo UJ	immediato[20, 10:1, 11, 19:12]				rd	codop	Istruzioni di salto incondizionato
Tipo U	immediato[31:12]				rd	codop	Formato caricamento stringhe di bit più significativi

Figura 2.19 Formati delle istruzioni RISC-V.